

LLM之RAG实战（六十一） | 构建一个深度思考的Agentic RAG Pipeline来解决复杂查询问题

ArronAI 2025-11-11 AI

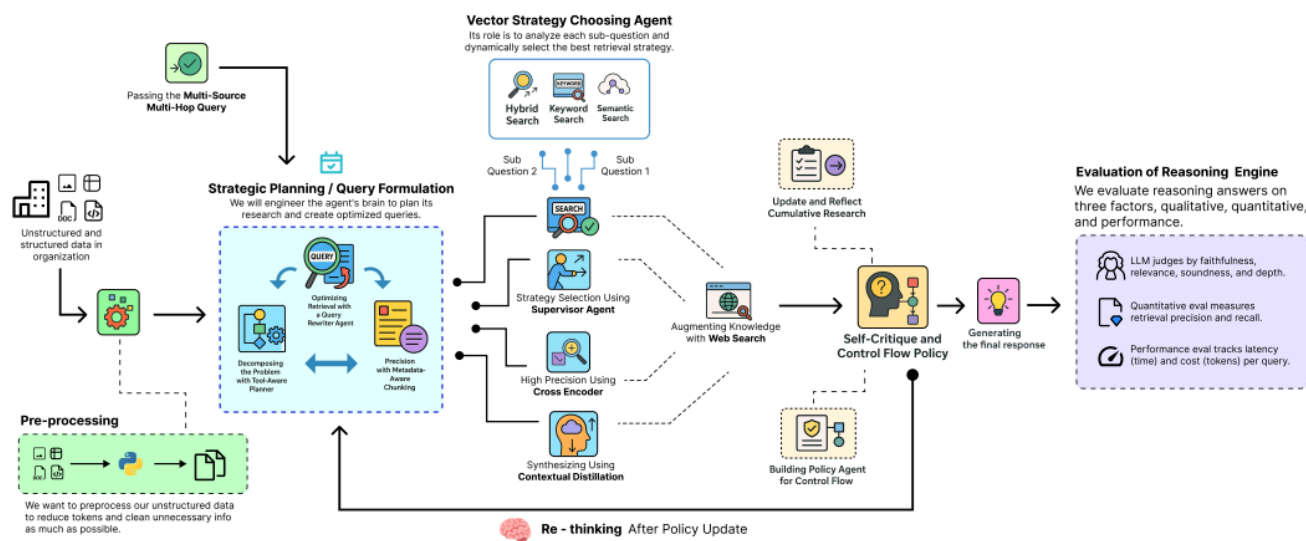
向量数据库 大模型 机器学习

推荐 机器学习平台

[了解详情 >](#)

机器学习平台是面向机器学习应用开发者，提供 WebIDE 和自定义训练等丰富建模工具、多框架高性...

RAG系统有时候会失败，不是因为 LLM 缺乏智能，而是因为其架构过于简单。人们试图用线性、一次许多复杂的查询需要推理、反思，以及关于何时采取行动的明智决策，就像我们面对问题时检索信息



1. 计划（Plan）：首先，代理将复杂的用户查询分解为结构化的、多步骤的研究计划，并决定每个步骤需要哪种工具（内部文档搜索或网络搜索）。
2. 检索（Retrieve）：对于每个步骤，supervisor会动态选择最佳搜索策略（向量、关键字或混合检索）来执行一个自适应的多阶段检索漏斗。
3. 细化（Refine）：然后使用高精度cross-encoder对初始结果进行重新排序，并使用代理提取最佳块并将其压缩到简洁的上下文中。

4. 反思 (Reflect): 每一步之后, Agent都会总结目前所有内容并更新其研究历史, 从而对问题建立累积的理解。
5. 批评 (Critique): 然后, 策略agent会检查历史信息, 做出战略决策: 要么继续下一步研究, 要么在遇到死胡同时修改计划, 要么结束研究。
6. 综合 (Synthesize): 研究完成后, 最终agent将从所有来源收集的所有证据综合成一个单一、全面且可引用的答案。

在这篇博客中, 我们将实现整个深度思考 RAG 管道, 并将其与基本的 RAG 管道进行比较, 以演示它如何解决复杂的多跳查询。

目录

- 配置环境
- 获取知识库
- 理解我们的多源、多跳查询
- 构建一个将会失败的浅层 RAG 管道
- 为中央代理系统定义 RAG 状态
- 战略规划和查询制定
- 使用工具感知规划器分解问题
- 使用查询重写agent优化检索
- 元数据感知分块的精度
- 创建多阶段检索漏斗
- 使用 Supervisor 动态选择策略
- 关键词和语义搜索的混合召回
- 使用cross-encoder重排序器实现高精度
- 使用上下文蒸馏进行合成
- 利用网络搜索增强知识
- 自我批评和控制流政策
- 更新并反映累积研究历史
- 构建控制流策略代理
- 定义图形节点
- 定义条件边
- 连接深度思考 RAG 系统
- 编译和可视化迭代工作流

- 运行深度思考管道
- 分析最终的高质量答案
- 比较结果
- 评估框架与分析结果
- 总结整个流程
- 利用马尔可夫决策过程（MDP）学习策略

一、配置环境

在开始编写深度 RAG Pipeline代码之前，我们需要配置环境。当我们开始开发管道并对其进行反复调

```
# Central Configuration Dictionary to manage all system parameters
config = {
    "data_dir": "./data",                # Directory to store raw
    "vector_store_dir": "./vector_store", # Directory to persist ou
    "llm_provider": "openai",            # The LLM provider we are
    "reasoning_llm": "gpt-4o",           # The powerful model for
    "fast_llm": "gpt-4o-mini",           # A faster, cheaper model
    "embedding_model": "text-embedding-3-small", # The model for creating
    "reranker_model": "cross-encoder/ms-marco-MiniLM-L-6-v2", # The model for
    "max_reasoning_iterations": 7,        # A safeguard to prevent
    "top_k_retrieval": 10,                # Number of documents for
    "top_n_rerank": 3,                   # Number of documents to
}
```

这些配置参数很容易理解，但有三个配置参数值得一提：

- `llm_provider`：这是我们使用的 LLM 提供商，在本例中是 OpenAI。我使用 OpenAI 是因为我们可以轻松地在 LangChain 中切换模型和提供商，但您也可以选择任何适合您需求的提供商，例如 Ollama。
- `reasoning_llm`：这必须是我们整个设置中最强大的，因为它将用于规划和综合。
- `fast_llm`：这应该是一个更快、更便宜的模型，因为它将用于更简单的任务，如baseline RAG。

现在需要导入管道使用的所需库，并将 api 密钥设置为环境变量，以避免在代码块中暴露它。

```
import os # For interacting with the operating system (e.g.,
import re # For regular expression operations, useful for te
import json # For working with JSON data
from getpass import getpass # To securely prompt for user input like API keys
from pprint import pprint # For pretty-printing Python objects, making them
import uuid # To generate unique identifiers
from typing import List, Dict, TypedDict, Literal, Optional # For type hints

# Helper function to securely set environment variables if they are not already
def _set_env(var: str):
    # Check if the environment variable is not already set
    if not os.environ.get(var):
        # If not, prompt the user to enter it securely
        os.environ[var] = getpass(f"Enter your {var}: ")

# Set the API keys for the services we will use
_set_env("OPENAI_API_KEY") # For accessing OpenAI models (GPT-4o, embeddings)
_set_env("LANGSMITH_API_KEY") # For tracing and debugging with LangSmith
_set_env("TAVILY_API_KEY") # For the web search tool

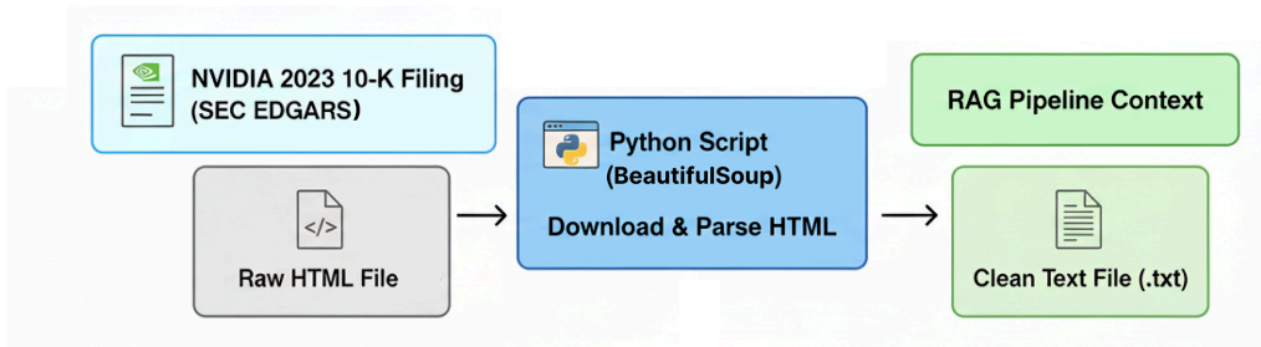
# Enable LangSmith tracing to get detailed logs and visualizations of our agent
os.environ["LANGSMITH_TRACING"] = "true"
# Define a project name in LangSmith to organize our runs
os.environ["LANGSMITH_PROJECT"] = "Advanced-Deep-Thinking-RAG"
```

这里还启用了 LangSmith 的追踪功能。当您使用复杂、循环工作流的代理系统时，它的追踪功能可

二、获取知识库

生产级 RAG 系统需要复杂且严苛的知识库才能真正展现其有效性。为此，我们将使用 NVIDIA 2021

Sourcing the Knowledge Base



首先，我们自定义一个函数直接从 SEC EDGAR 数据库下载 10-K 文件，解析原始 [HTML](#)，并将其转

```
import requests # For making HTTP requests to download the document
from bs4 import BeautifulSoup # A powerful library for parsing HTML and XML
from langchain.docstore.document import Document # LangChain's standard data structure

def download_and_parse_10k(url, doc_path_raw, doc_path_clean):
    # Check if the cleaned file already exists to avoid re-downloading
    if os.path.exists(doc_path_clean):
        print(f"Cleaned 10-K file already exists at: {doc_path_clean}")
        return

    print(f"Downloading 10-K filing from {url}...")
    # Set a User-Agent header to mimic a browser, as some servers block scripts
    headers = {'User-Agent': 'Mozilla/5.0'}
    # Make the GET request to the URL
    response = requests.get(url, headers=headers)
    # Raise an error if the download fails (e.g., 404 Not Found)
    response.raise_for_status()

    # Save the raw HTML content to a file for inspection
    with open(doc_path_raw, 'w', encoding='utf-8') as f:
        f.write(response.text)
        print(f"Raw document saved to {doc_path_raw}")

    # Use BeautifulSoup to parse and clean the HTML content
    soup = BeautifulSoup(response.content, 'html.parser')

    # Extract text from common HTML tags, attempting to preserve paragraph structure
    text = ''
    for p in soup.find_all(['p', 'div', 'span']):
        # Get the text from each tag, stripping extra whitespace, and add newlines
        text += p.get_text(strip=True) + '\n\n'
```

```

# Use regex to clean up excessive newlines and spaces for a cleaner final
clean_text = re.sub(r'\n{3,}', '\n\n', text).strip() # Collapse 3+ newlines
clean_text = re.sub(r'\s{2,}', ' ', clean_text).strip() # Collapse 2+ spaces

# Save the final cleaned text to a .txt file
with open(doc_path_clean, 'w', encoding='utf-8') as f:
    f.write(clean_text)
print(f"Cleaned text content extracted and saved to {doc_path_clean}")

```

代码很容易理解，使用 `beautifulsoup4` 来解析 [HTML](#) 内容并提取文本。它将帮助我们轻松浏览 [HTML](#)

现在，让我们执行它并看看它是如何工作的。

```

print("Downloading and parsing NVIDIA's 2023 10-K filing...")
# Execute the download and parsing function
download_and_parse_10k(url_10k, doc_path_raw, doc_path_clean)

# Open the cleaned file and print a sample to verify the result
with open(doc_path_clean, 'r', encoding='utf-8') as f:
    print("\n--- Sample content from cleaned 10-K ---")
    print(f.read(1000) + "...")

#### OUTPUT ####
Downloading and parsing NVIDIA 2023 10-K filing...
Successfully downloaded 10-K filing from https://www.sec.gov/Archives/edgar/c
Raw document saved to ./data/nvda_10k_2023_raw.html
Cleaned text content extracted and saved to ./data/nvda_10k_2023_clean.txt

# --- Sample content from cleaned 10-K ---
Item 1. Business.
OVERVIEW
NVIDIA is the pioneer of accelerated computing. We are a full-stack computi
Founded in 1993, we started as a PC graphics chip company, inventing the gra
The programmability of our GPUs made them ...

```

我们只是调用这个函数将所有内容存储在一个 `txt` 文件中，该文件将作为我们的 `rag` 管道的上下文。

当我们运行上述代码时，您可以看到它开始为我们下载报告，并且我们可以看到下载内容的样本是什么样

三、理解我们的多源、多跳查询

为了测试我们实施的管道并将其与基本 RAG 进行比较，我们需要使用一个非常复杂的查询，涵盖我们

Our Complex Query:

"Based on NVIDIA's 2023 10-K filing, identify their key risks related to competition. Then, find recent news (post-filing, from 2024) about AMD's AI chip strategy and explain how this new strategy directly addresses or exacerbates one of NVIDIA's stated risks."

让我们分析一下为什么这个查询对于标准 RAG 管道来说如此困难：

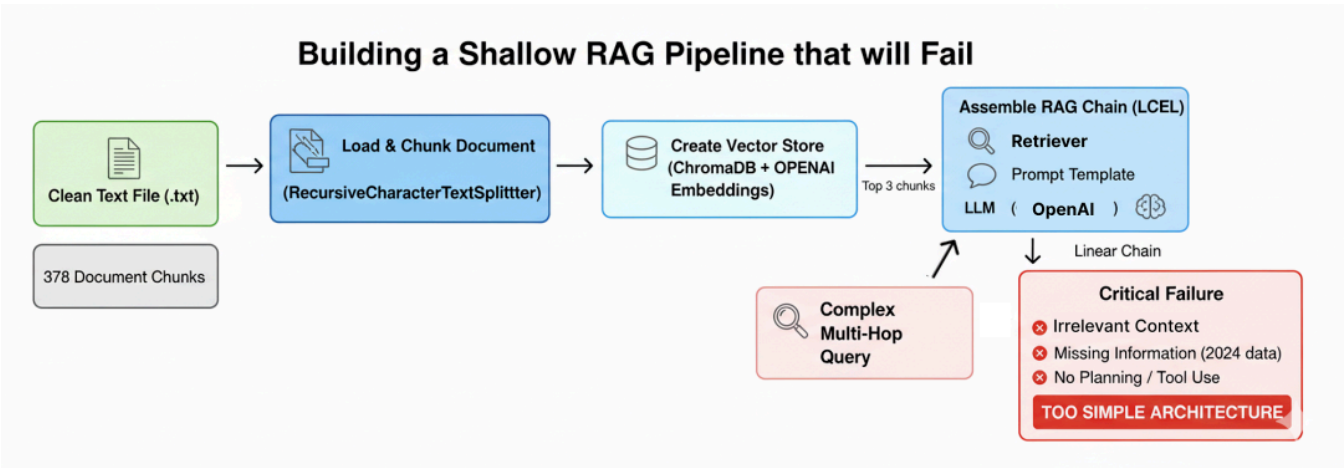
1. 多跳推理：系统必须首先识别风险，然后找到 AMD 消息，最后将两者综合起来。
2. 多源知识： 所需信息存在于两个完全不同的地方。风险信息存在于我们静态的内部文档（10-K）中，而 AMD 新闻则位于外部，需要访问实时网络。
3. 综合与分析： 该查询并非要求简单地列出事实。它要求解释一组事实如何使另一组事实恶化，这项任务需要真正的综合能力。

在下一节中，我们将实现基本的 RAG 管道，实际上看看简单的 RAG 是如何失败的。

四、构建一个将会失败的浅层 RAG 管道

现在我们已经配置好了环境，并且准备好了具有挑战性的知识库，下一步就是构建一个标准的原生 RA

首先构建最简单的解决方案，我们可以对其运行复杂的查询，并观察它失败的原因和方式。



- 加载和分块文档：我们将提取已清理的 10-K 文件并将其拆分成小的、固定大小的块，这是一种常见但语义上简单的方法。
- 创建向量存储：然后我们将嵌入这些块并将它们索引到 ChromaDB 向量存储中，以实现基本的语义搜索。
- 组装 RAG 链：我们将使用 LangChain 表达语言 (LCEL)将我们的检索器、提示模板和 LLM 连接成一个线性管道。
- 演示复杂查询：我们将针对这个简单的系统执行多跳、多源查询，并分析其不充分的响应。

首先，我们需要加载已清理的文档并将其拆分。我们将使用 LangChain 生态系统中的标准工具 RecursiveCharacterTextSplitter 。

```
from langchain_community.document_loaders import TextLoader # A simple loader
from langchain.text_splitter import RecursiveCharacterTextSplitter # A standard splitter

print("Loading and chunking the document...")
# Initialize the loader with the path to our cleaned 10-K file
loader = TextLoader(doc_path_clean, encoding='utf-8')
# Load the document into memory
documents = loader.load()

# Initialize the text splitter with a defined chunk size and overlap
# chunk_size=1000: Each chunk will be approximately 1000 characters long.
# chunk_overlap=150: Each chunk will share 150 characters with the previous chunk.
text_splitter = RecursiveCharacterTextSplitter(chunk_size=1000, chunk_overlap=150)
# Split the loaded document into smaller, manageable chunks
doc_chunks = text_splitter.split_documents(documents)

print(f"Document loaded and split into {len(doc_chunks)} chunks.")

#### OUTPUT ####
Loading and chunking the document...
Document loaded and split into 378 chunks.
```

我们的主文档中有 378 数据块，下一步创建向量嵌入并将其存储在数据库中。我们将使用 ChromaDB

```
from langchain_community.vectorstores import Chroma # The vector store we will use
from langchain_openai import OpenAIEmbeddings # The function to create embeddings
```



```

print("Creating baseline vector store...")
# Initialize the embedding function using the model specified in our config
embedding_function = OpenAIEmbeddings(model=config['embedding_model'])

# Create the Chroma vector store from our document chunks
# This process takes each chunk, creates an embedding for it, and indexes it.
baseline_vector_store = Chroma.from_documents(
    documents=doc_chunks,
    embedding=embedding_function
)
# Create a retriever from the vector store
# The retriever is the component that will actually perform the search.
# search_kwargs={"k": 3}: This tells the retriever to return the top 3 most relevant documents.
baseline_retriever = baseline_vector_store.as_retriever(search_kwargs={"k": 3})

print(f"Vector store created with {baseline_vector_store._collection.count()} embeddings.")

```

OUTPUT

```

Creating baseline vector store...
Vector store created with 378 embeddings.

```

`Chroma.from_documents` 将所有向量存储在一个可搜索的索引中。最后一步是使用 `LangChain`

该链将定义数据的线性流：从用户的问题到检索器，再到提示，最后到 LLM。

```

from langchain_core.prompts import ChatPromptTemplate # For creating prompt templates
from langchain_openai import ChatOpenAI # The OpenAI chat model interface
from langchain_core.runnables import RunnablePassthrough # A tool to pass input directly to the LLM
from langchain_core.output_parsers import StrOutputParser # To parse the LLM output

# This template instructs the LLM on how to behave.
# {context}: This is where we will inject the content from our retrieved documents.
# {question}: This is where the user's original question will go.
template = """You are an AI financial analyst. Answer the question based only on the following context:
{context}

Question: {question}
"""

prompt = ChatPromptTemplate.from_template(template)
# We use our 'fast_llm' for this simple task, as defined in our config
llm = ChatOpenAI(model=config["fast_llm"], temperature=0)

# A helper function to format the list of retrieved documents into a single string
def format_docs(docs):

```

```

        return "\n\n---\n\n".join(doc.page_content for doc in docs)

# The complete RAG chain defined using LCEL's pipe (|) syntax
baseline_rag_chain = (
    # The first step is a dictionary that defines the inputs to our prompt
    {"context": baseline_retriever | format_docs, "question": RunnablePassthru}
    # The context is generated by taking the question, passing it to the retriever
    # The original question is passed through unchanged
    | prompt # The dictionary is then passed to the prompt template
    | llm # The formatted prompt is passed to the language model
    | StrOutputParser() # The LLM's output message is parsed into a string
)

```

首先，定义了提示词模板，它的 `context` 键由子链填充，输入问题来自 `baseline\_retriever`，

让我们运行这个简单的管道并了解它在哪里失败了。

```

from rich.console import Console # For pretty-printing output with markdown
from rich.markdown import Markdown

# Initialize the rich console for better output formatting
console = Console()

# Our complex, multi-hop, multi-source query
complex_query_adv = "Based on NVIDIA's 2023 10-K filing, identify their key i

print("Executing complex query on the baseline RAG chain...")
# Invoke the chain with our challenging query
baseline_result = baseline_rag_chain.invoke(complex_query_adv)

console.print("\n--- BASELINE RAG FAILED OUTPUT ---")
# Print the result using markdown formatting for readability
console.print(Markdown(baseline_result))

```

当您运行上述代码时，我们会得到以下输出。

```

#### OUTPUT ####
Executing complex query on the baseline RAG chain...

```

--- BASELINE RAG FAILED OUTPUT ---

Based on the provided context, NVIDIA operates in an intensely competitive semiconductor industry and faces competition from companies like AMD. The context mentions that the industry is characterized by rapid technological change. However, the

分析一下该RAG 管道失败的三个方面：

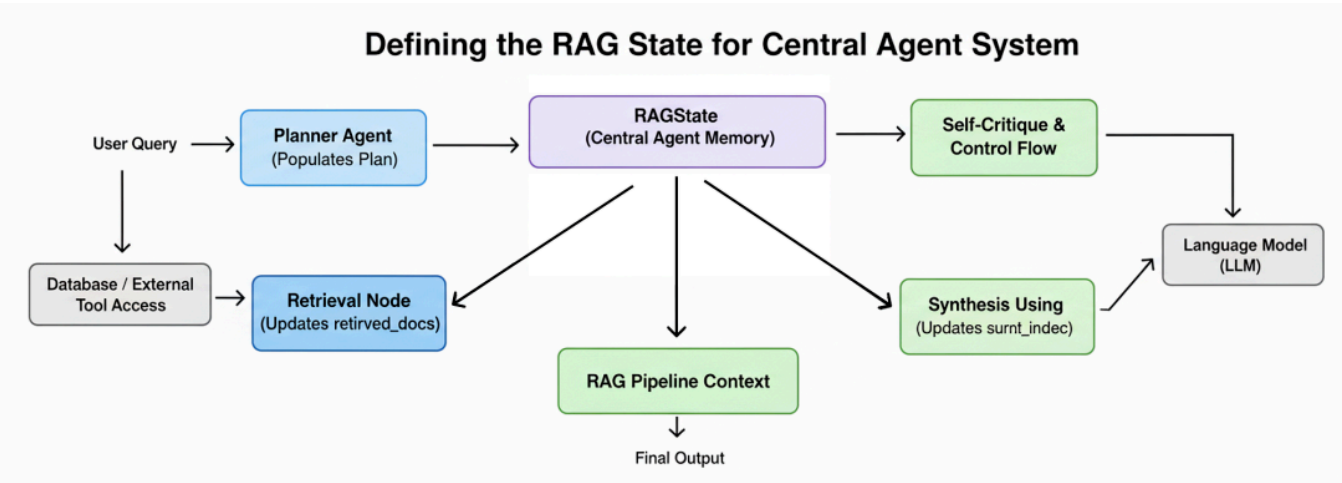
- 不相关的上下文：Retriever 抓取了 “NVIDIA” 、 “competition” 和 “AMD” 的一般信息，但没有抓取到 2024 年 AMD 战略的具体细节。
- 信息缺失：关键缺陷在于 2023 年的数据无法涵盖 2024 年的事件。系统没有意识到自己缺少关键信息。
- 缺乏规划或工具使用：将复杂的查询视为简单的查询，无法将其分解为多个步骤，也无法使用网页搜索等工具来填补空白。

该系统的失败并非因为LLM太笨，而是因为其架构过于简单 。它试图用一个线性的、一次性的过程：

我们已经了解了基本 RAG 管道的问题，接下来让我们看看深度思考RAG是如何解决好复杂查询的。

五、为Agent系统定义RAGState

要构建推理agent，我们首先需要一种方法来管理它的状态。在我们简单的 RAG 链中，每个步骤都：



RAGState 将充当中央内存，在 LangGraph 工作流程中的每个节点之间传递。为了构建它，我们将

我们希望定义智能体计划的原子单元。每个 `Step` 不仅要包含一个需要回答的问题，还要包含其背后的

```
from langchain_core.documents import Document
from langchain_core.pydantic_v1 import BaseModel, Field

# Pydantic model for a single step in the agent's reasoning plan
class Step(BaseModel):
    # A specific, answerable sub-question for this research step
    sub_question: str = Field(description="A specific, answerable question for this step")
    # The agent's justification for why this step is necessary
    justification: str = Field(description="A brief explanation of why this step is necessary")
    # The specific tool to use for this step: either internal document search or web search
    tool: Literal["search_10k", "search_web"] = Field(description="The tool to use for this step")
    # A list of critical keywords to improve the accuracy of the search
    keywords: List[str] = Field(description="A list of critical keywords for this step")
    # (Optional) A likely document section to perform a more targeted, filtered search
    document_section: Optional[str] = Field(description="A likely document section to search")
```

`Step`类使用了 `Pydantic BaseModel`，定义了`Planner Agent`严格执行的规范。`tool` `tool: Li`

这种结构化的输出比尝试解析自然语言计划要可靠得多。

现在我们已经定义了一个 `Step`，我们需要一个容器来保存整个步骤序列。我们将创建一个 `Plan` 类

```
# Pydantic model for the overall plan, which is a list of individual steps
class Plan(BaseModel):
    # A list of Step objects that outlines the full research plan
    steps: List[Step] = Field(description="A detailed, multi-step plan to answer the question")
```

我们编写了一个 `Plan` 类，它将为整个研究过程提供结构。当我们调用 `Planner Agent` 时，我们

接下来，当我们的智能体执行其计划时，它需要一种方法来记住它所学到的知识。我们将定义一个 `Pa`

```
# A TypedDict to store the results of a completed step in our research history
class PastStep(TypedDict):
    step_index: int          # The index of the completed step (e.g., 1,
    sub_question: str        # The sub-question that was addressed in this step
    retrieved_docs: List[Document] # The precise documents retrieved and reranked
    summary: str             # The agent's one-sentence summary of the findings
```

PastStep 结构对于智能体的自我批评循环至关重要。每完成一步，我们都会填充其中一个字典并将其添加到历史中。

最后，我们将所有这些部分整合到主 RAGState 字典中。它是贯穿整个图的核心对象，包含原始查询和计划。

```
# The main state dictionary that will be passed between all nodes in our LangChain graph
class RAGState(TypedDict):
    original_question: str    # The initial, complex query from the user that we want to answer
    plan: Plan                # The multi-step plan generated by the Planner
    past_steps: List[PastStep] # A cumulative history of completed research steps
    current_step_index: int    # The index of the current step in the plan being executed
    retrieved_docs: List[Document] # Documents retrieved in the current step
    reranked_docs: List[Document] # Documents after precision reranking in the current step
    synthesized_context: str    # The concise, distilled context generated from the current step's docs
    final_answer: str          # The final, synthesized answer to the user's question
```

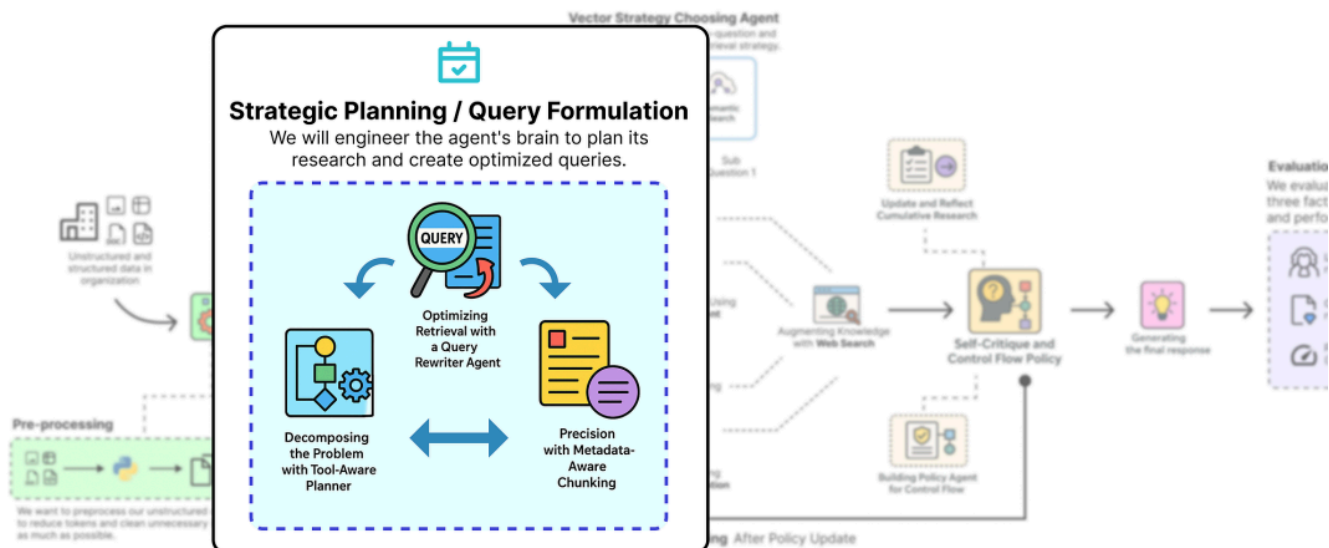
这个 RAGState(TypedDict) 是我们agent的完整大脑。图中的每个节点都会接收这个字典作为输入并返回更新后的字典。

例如，plan_node将填充 plan 字段，retrieval_node将填充 retrieved_docs 字段，依此类推。

现在，我们已经定义了agent的记忆蓝图，可以构建系统的第一个认知组件：填充此状态的规划agent。

六、战略规划和查询制定

定义好 RAGState 后，我们现在可以构建agent的第一个、可以说是最关键的认知组件：规划能力，它负责制定战略规划和查询制定。

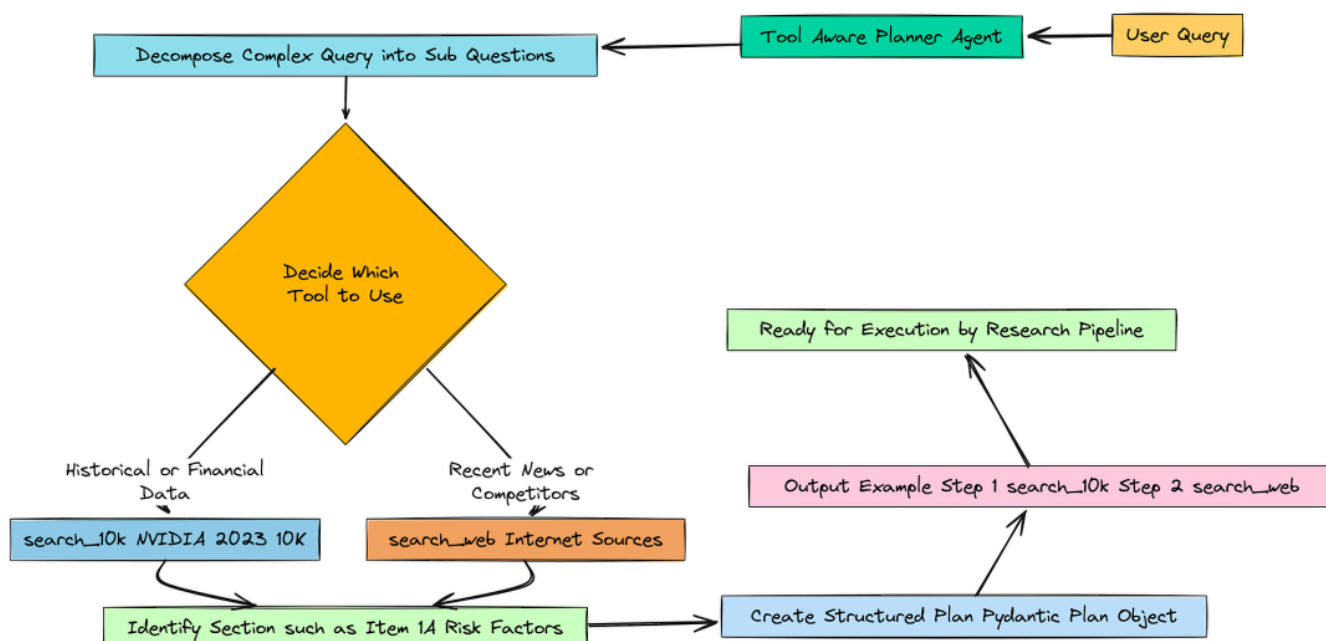


本节分为三个关键工程步骤：

- 工具感知规划器：我们将构建一个由 LLM 驱动的agent，其唯一工作是将用户的查询分解为结构化的 Plan 对象，并决定每个步骤使用哪种工具。
- 查询重写器：我们将创建一个专门的agent，将规划器的简单子问题转换为高效、优化的搜索查询。
- 元数据感知分块：我们将重新处理源文档以添加部分级元数据，这是实现高精度、过滤检索的关键步骤。

6.1 使用工具感知规划器分解问题

我们要构建我们的行动大脑，当这个大脑遇到一个复杂的问题时，它需要做的第一件事就是制定一个任务



我们不能直接把整个问题扔到数据库中，然后指望它能给出最好的结果。我们需要教会代理如何将问题分解成更小、更易于处理的部分。

为此，我们将创建一个专用的规划agent，我们需要给它一组非常清晰的指令或提示，明确地告诉它它的

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from rich.pretty import pprint as rprint

# The system prompt that instructs the LLM how to behave as a planner
planner_prompt = ChatPromptTemplate.from_messages([
    ("system", """You are an expert research planner. Your task is to create
You have two tools available:
1. `search_10k`: Use this to search for information within NVIDIA's 2023 10-K
2. `search_web`: Use this to search the public internet for recent news, comp
Decompose the user's query into a series of simple, sequential sub-questions.
For `search_10k` steps, also identify the most likely section of the 10-K (e.
It is critical to use the exact section titles found in a 10-K filing where
    ("human", "User Query: {question}") # The user's original, complex query
])
```

我们实际上是在赋予 LLM 一个新的角色：一位专家级的研究规划师。我们明确地告诉它可以使用的两

我们不仅要求它制定计划，还要求它创建一个直接映射到我们已经构建的能力的计划。

现在我们可以启动推理模型并将其与提示链接在一起，这里非常重要的一步是告诉 LLM 它的最终输出

```
# Initialize our powerful reasoning model, as defined in the config
reasoning_llm = ChatOpenAI(model=config["reasoning_llm"], temperature=0)

# Create the planner agent by piping the prompt to the LLM and instructing it
planner_agent = planner_prompt | reasoning_llm.with_structured_output(Plan)
print("Tool-Aware Planner Agent created successfully.")

# Let's test the planner agent with our complex query to see its output
print("\n--- Testing Planner Agent ---")
test_plan = planner_agent.invoke({"question": complex_query_adv})
```

```
s# Use rich's pretty print for a clean, readable display of the Pydantic object
rprint(test_plan)
```

我们将 `planner\_prompt` 传递给强大的 `reasoning\_llm`，然后使用 `.with\_structured\_output`。

让我们看看使用挑战查询进行测试时的输出。

```
#### OUTPUT ####
```

```
Tool-Aware Planner Agent created successfully.
```

```
--- Testing Planner Agent ---
```

```
Plan(
  steps=[
    Step(
      sub_question="What are the key risks",
      document_section='Item 1A. Risk Factors'
    ),
    Step(
      sub_question="What are the recent news and developments in AMD's",
      justification="This step requires finding up-to-date, external information",
      tool='search_web',
      keywords=['AMD', 'AI chip strategy', '2024', 'MI300X', 'Instinct MI300X'],
      document_section=None
    )
  ]
)
```

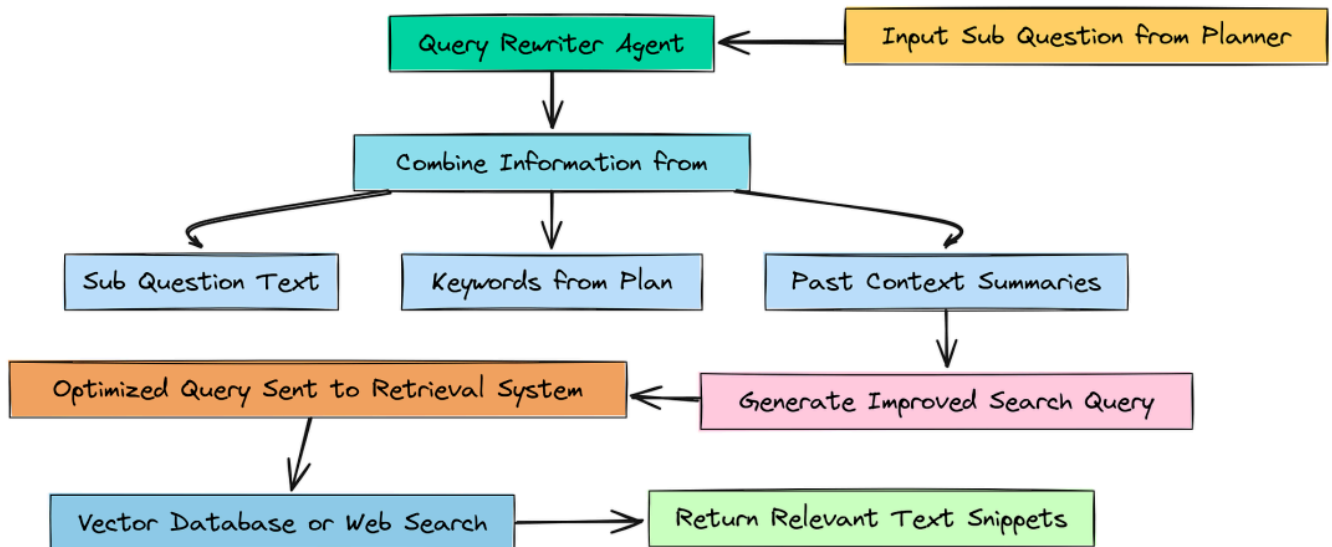
如果我们查看输出，就会发现agent不仅仅给出了一个模糊的计划，它还生成了一个结构化的 `Plan` 对象。

1. 对于第一部分，它知道答案在 10-K 中并选择了 `search_10k` 工具，甚至正确猜出了正确的文档部分。
2. 对于第二部分，它知道“2024 年的新闻”不可能出现在 2023 年的文档中，并正确地选择了 `search_web` 工具。这标志着我们的管道至少在思考过程中能够给出令人满意的结果。

6.2 使用查询重写代理优化检索

所以，基本上我们有一个包含良好子问题的计划。

但像“风险是什么？”这样的问题并不是一个很好的搜索查询。它太笼统了。搜索引擎，无论是矢量数据库还是网页搜索，最适合处理具体的、关键词丰富的查询。



为了解决这个问题，我们将构建另一个小型的专用agent：查询重写器。它的唯一工作就是获取当前步

首先，让我们为这个新代理设计提示。

```
from langchain_core.output_parsers import StrOutputParser # To parse the LLM

# The prompt for our query rewriter, instructing it to act as a search expert
query_rewriter_prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a search query optimization expert. Your task is to
    The rewritten query should be specific, use terminology likely to be found in
    ("human", "Current sub-question: {sub_question}\n\nRelevant keywords from
```

我们基本上是在告诉这个agent，让它像一个搜索查询优化专家一样工作。我们给它提供了三条信息：当前

现在我们可以启动这个agent了。这是一个简单的链，因为我们只需要一个字符串作为输出。

```
# Create the agent by piping the prompt to our reasoning LLM and a string output parser
query_rewriter_agent = query_rewriter_prompt | reasoning_llm | StrOutputParser
print("Query Rewriter Agent created successfully.")
```

```

# Let's test the rewriter agent. We'll pretend we've already completed the f:
print("\n--- Testing Query Rewriter Agent ---")

# Let's imagine we are at a final synthesis step that needs context from the
test_sub_q = "How does AMD's 2024 AI chip strategy potentially exacerbate the
test_keywords = ['impact', 'threaten', 'competitive pressure', 'market share

# We create some mock "past context" to simulate what the agent would know at
test_past_context = "Step 1 Summary: NVIDIA's 10-K lists intense competition

# Invoke the agent with our test data
rewritten_q = query_rewriter_agent.invoke({
    "sub_question": test_sub_q,
    "keywords": test_keywords,
    "past_context": test_past_context
})

print(f"Original sub-question: {test_sub_q}")
print(f"Rewritten Search Query: {rewritten_q}")

```

为了正确测试这一点，我们必须模拟一个真实的场景。我们创建一个 `test\_past\_context` 字符串

我们来看看结果。

```

#### OUTPUT ####
Query Rewriter Agent created successfully.

--- Testing Query Rewriter Agent ---
Original sub-question: How does AMD 2024 AI chip strategy potentially exacerbate the
Rewritten Search Query: analysis of how AMD 2024 AI chip strategy, including

```

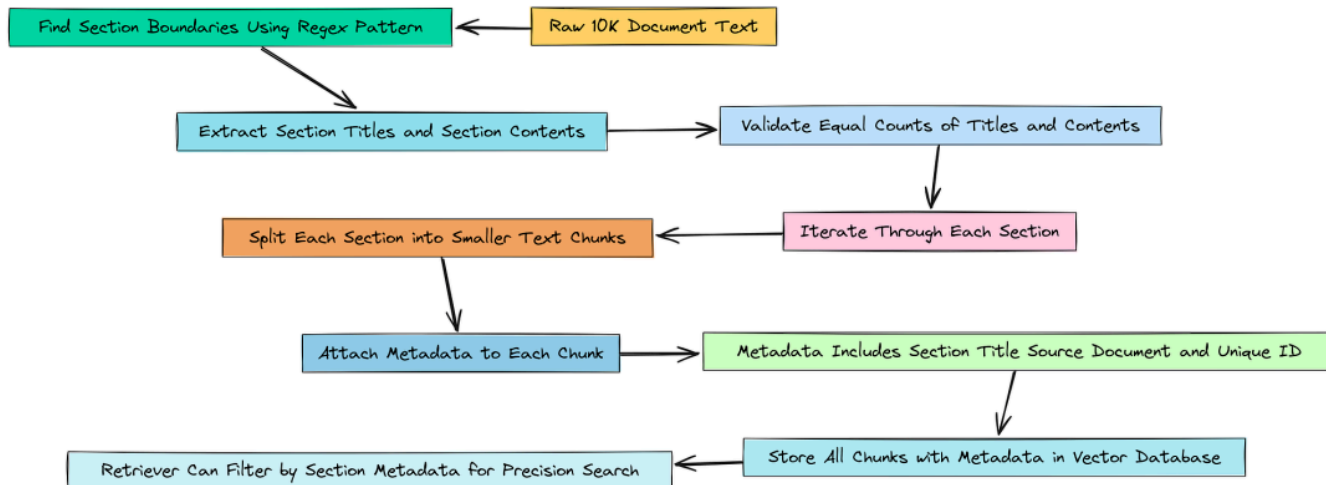
最初的问题是针对分析师的，他重写了查询语句，用于搜索引擎。它被分配了特定的术语，例如 “MI300”

这样的查询更有可能检索到正确的文档，从而使我们的整个系统更加准确和高效。这个重写步骤将成为

6.3 元数据感知分块的精度

规划agent给了我们一个很好的机会。它不仅仅是告诉我们“发现风险”，还给了我们一个提示：在

但目前，我们的检索器无法使用该提示。我们的向量存储只是一个包含 378 个文本块的扁平大列表。它根本不知道“section”是什么。



我们需要解决这个问题。我们将从头开始重建文档块。这次，对于我们创建的每个块，我们将添加一个标

首先，我们需要一种编程方式来查找原始文本文件中每个部分的起始位置。查看文档，我们可以看到一

```
# This regex is designed to find section titles like 'ITEM 1A.' or 'ITEM 7.'
# It looks for the word 'ITEM', followed by a space, a number, an optional letter
# The `re.IGNORECASE | re.DOTALL` flags make the search case-insensitive and
section_pattern = r"(ITEM\\s+\\d[A-Z]?\\.\\s*\\.*)?(?=\\nITEM\\s+\\d[A-Z]?\\.\\s*\\.*)"
```

我们基本上是在创建一个模式，作为我们的文本段检测器。它应该被设计得足够灵活，能够捕捉不同的

现在我们可以使用此模式将文档分成两个单独的列表：一个仅包含章节标题，另一个包含每个章节内的内

```
# We'll work with the raw text loaded earlier from our Document object
raw_text = documents[0].page_content

# Use re.findall to apply our pattern and extract all section titles into a list
```

```

section_titles = re.findall(section_pattern, raw_text, re.IGNORECASE | re.DOTALL)

# A quick cleanup step to remove any extra whitespace or newlines from the titles
section_titles = [title.strip().replace('\n', ' ') for title in section_titles]

# Now, use re.split to break the document apart at each point where a section title occurs
sections_content = re.split(section_pattern, raw_text, flags=re.IGNORECASE | re.DOTALL)

# The split results in a list with titles and content mixed, so we filter it to get just the content
sections_content = [content.strip() for content in sections_content if content.strip()]

print(f"Identified {len(section_titles)} document sections.")

# This is a crucial sanity check: if the number of titles doesn't match the number of content blocks,
assert len(section_titles) == len(sections_content), "Mismatch between titles and content blocks"

```

这是一种解析半结构化文档的非常有效的方法。我们使用了两次正则表达式：一次是获取所有章节标题的列表，另一次是获取每个标题对应的内容块。

好了，现在我们有了一些部分：一个标题列表和一个对应的内容列表。现在我们可以循环遍历它们，并将每个内容块拆分成更小的块。

```

import uuid # We'll use this to give each chunk a unique ID, which is good practice

# This list will hold our new, metadata-rich document chunks
doc_chunks_with_metadata = []

# Loop through each section's content along with its title using enumerate
for i, content in enumerate(sections_content):
    # Get the corresponding title for the current content block
    section_title = section_titles[i]
    # Use the same text splitter as before, but this time, we run it ONLY on the content
    section_chunks = text_splitter.split_text(content)

    # Now, loop through the smaller chunks created from this one section
    for chunk in section_chunks:
        # Generate a unique ID for this specific chunk
        chunk_id = str(uuid.uuid4())
        # Create a new LangChain Document object for the chunk
        doc_chunks_with_metadata.append(
            Document(
                page_content=chunk,
                # This is the most important part: we attach the metadata
                metadata={
                    "section": section_title, # The section this chunk belongs to
                    "source_doc": doc_path_clean, # Where the document came from
                    "id": chunk_id # The unique ID for this chunk
                }
            )
        )

```

```
)  
)
```

```
print(f"Created {len(doc_chunks_with_metadata)} chunks with section metadata.  
print("\n--- Sample Chunk with Metadata ---")
```

```
# To prove it worked, let's find a chunk that we know should be in the 'Risk  
sample_chunk = next(c for c in doc_chunks_with_metadata if "Risk Factors" in  
print(sample_chunk)
```

这是我们升级的核心。我们逐一迭代每个部分。对于每个部分，我们创建文本块。但在将它们添加到最

让我们看一下输出并看看有什么区别。

```
#### OUTPUT ####
```

```
Processing document and adding metadata...
```

```
Identified 22 document sections.
```

```
Created 381 chunks with section metadata.
```

```
--- Sample Chunk with Metadata ---
```

```
Document(  
|
```

```
|   page_content='Our industry is intensely competitive. We operate in the s  
|   metadata={
```

```
|       |
```

```
|       |   'section': 'Item 1A. Risk Factors.',
```

```
|       |   'source_doc': './data/nvda_10k_2023_clean.txt',
```

```
|       |   'id': '...'
```

```
|       |
```

```
|   }
```

```
)
```

看看那个 `metadata` 块。我们之前提到的那段文本现在附加了一段上下文： `'section': 'Item`

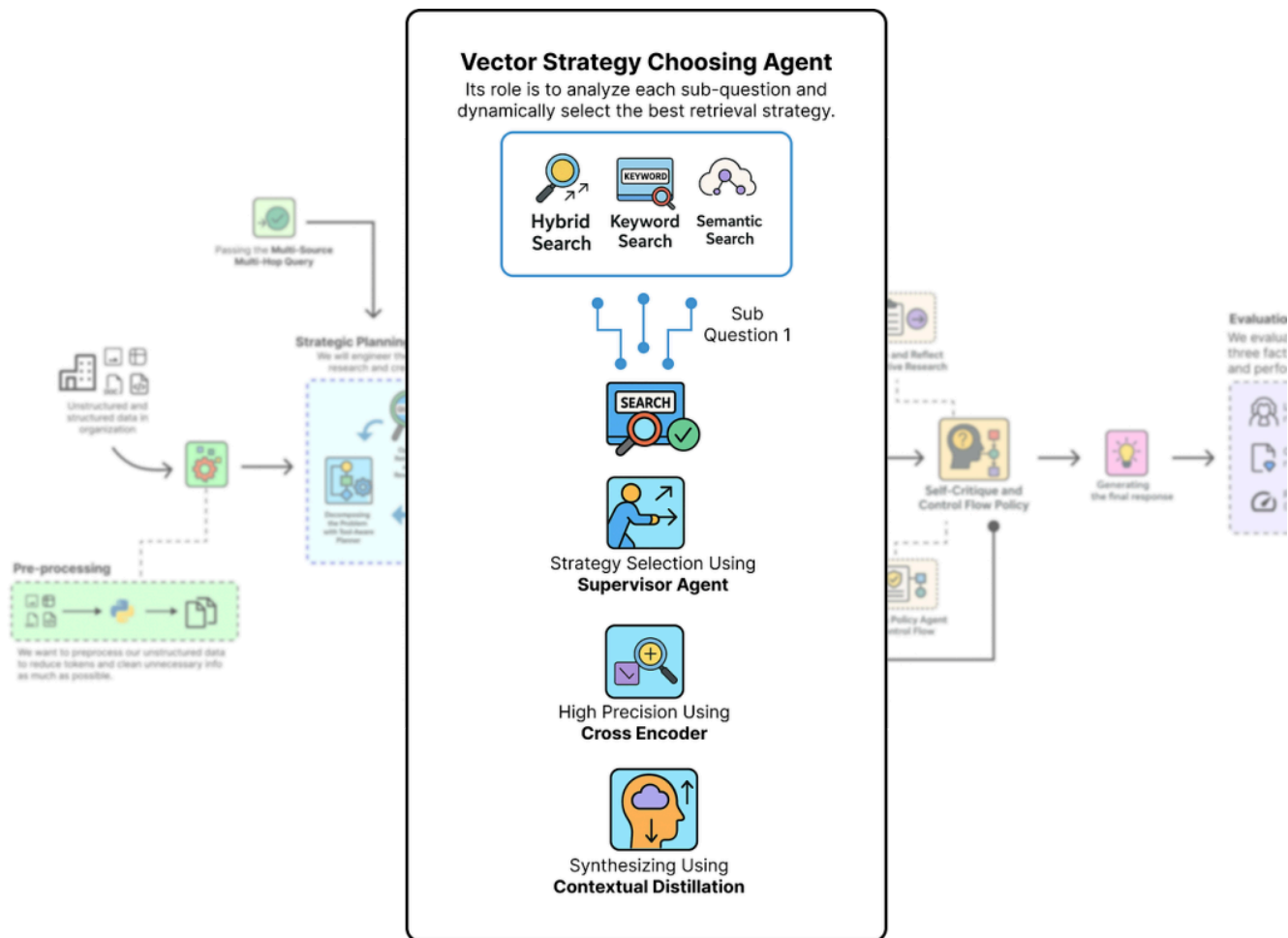
现在，当我们的agent需要查找风险时，它可以告诉检索器：“嘿，不要搜索所有 381 个块。只需提

这个简单的改变将我们的检索器从钝器转变为手术工具，这是构建真正生产级 RAG 系统的关键原则。

七、创建多阶段检索漏斗

到目前为止，我们已经设计了一个智能规划器，并用元数据丰富了我们的文档。现在，我们准备构建系统简单的一次性语义搜索已经不够了。对于生产级代理，我们需要一个自适应且多阶段的检索过程。

我们将把检索过程设计成一个漏斗，其中每个阶段都会细化前一个阶段的结果：

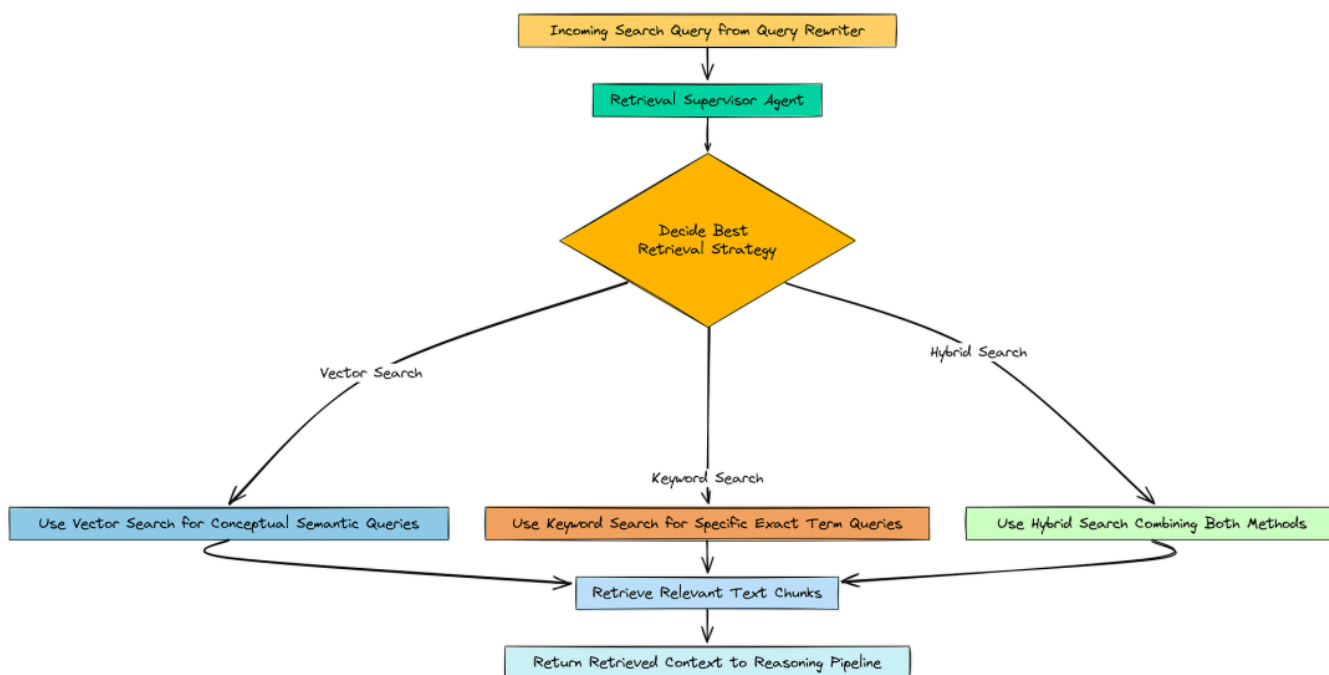


- 检索Supervisor：构建一个新的Supervisor代理，充当动态路由器，分析每个子问题并选择最佳搜索策略（向量、关键字或混合）。
- 第 1 阶段（广泛回忆）：Supervisor可以选择的不同检索策略，重点是广泛收集所有可能相关的文档。
- 第 2 阶段（高精度）：使用 Cross-Encoder 模型对初始结果进行重新排序，丢弃噪音并将最相关的文档排序到最前面。
- 第 3 阶段（综合）：最后，我们将创建一个 Distiller Agent，将排名靠前的文档压缩成一个简洁的段落，供我们的下游agent使用。

7.1 使用 Supervisor 动态选择策略

并非所有搜索查询都是一样的。像“计算和网络”部分的收入是多少？这样的问题包含具体、精确的术语

公司对市场竞争的看法是什么？这还是个概念问题。基于语义向量的搜索会更好。



我们不会采用硬编码一个策略，而是构建一个小型智能代理——检索主管（Retrieval Supervisor）

首先，我们需要定义主管可能做出的决策。我们将使用 Pydantic BaseModel 来构建其输出。

```
class RetrievalDecision(BaseModel):
    # The chosen retrieval strategy. Must be one of these three options.
    strategy: Literal["vector_search", "keyword_search", "hybrid_search"]
    # The agent's justification for its choice.
    justification: str
```

监管者必须从这三种策略中选择一种，并解释其理由。这使得其决策过程透明可靠。

现在，让我们创建指导该代理行为的提示。

```
retrieval_supervisor_prompt = ChatPromptTemplate.from_messages([
    ("system", """"You are a retrieval strategy expert. Based on the user's q
```

You have three options:

1. ``vector_search``: Best for conceptual, semantic, or similarity-based queries
 2. ``keyword_search``: Best for queries with specific, exact terms, names, or categories
 3. ``hybrid_search``: A good default that combines both, but may be less precise
- ```
 ("human", "User Query: {sub_question}") # The rewritten search query will be
])
```

我们在这里创建了一个非常直接的提示，告诉 LLM 它的角色是检索策略专家，并清楚地解释其每个可

最后，我们可以组装我们的主管代理。

```
Create the agent by piping our prompt to the reasoning LLM and structuring
retrieval_supervisor_agent = retrieval_supervisor_prompt | reasoning_llm.with_structured_output(RetrievalDecision)
print("Retrieval Supervisor Agent created.")

Let's test it with two different types of queries to see how it behaves
print("\n--- Testing Retrieval Supervisor Agent ---")
query1 = "revenue growth for the Compute & Networking segment in fiscal year 2023"
decision1 = retrieval_supervisor_agent.invoke({"sub_question": query1})

print(f"Query: '{query1}'")
print(f"Decision: {decision1.strategy}, Justification: {decision1.justification}")

query2 = "general sentiment about market competition and technological innovation in the AI sector"
decision2 = retrieval_supervisor_agent.invoke({"sub_question": query2})
print(f"\nQuery: '{query2}'")
print(f"Decision: {decision2.strategy}, Justification: {decision2.justification}")
```

使用`.with_structured_output(RetrievalDecision)`把所有东西连在一起，确保我们从 LLM

```
OUTPUT
```

```
Retrieval Supervisor Agent created.
```

```
--- Testing Retrieval Supervisor Agent ---
```

```
Query: 'revenue growth for the Compute & Networking segment in fiscal year 2023'
Decision: keyword_search, Justification: The query contains specific keywords related to revenue growth and fiscal year 2023.
```



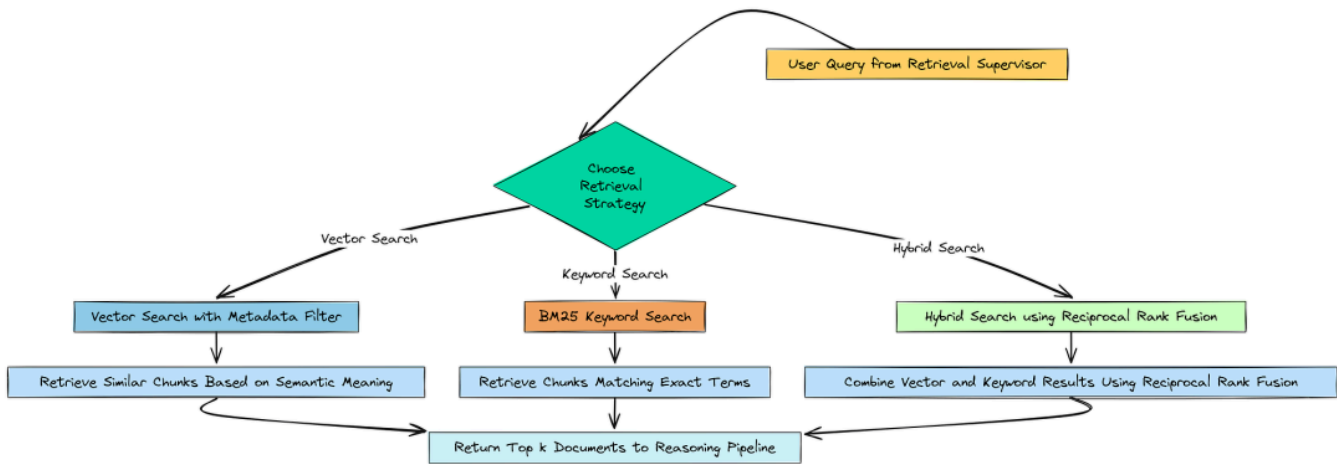
Query: 'general sentiment about market competition and technological innovation'  
Decision: vector\_search, Justification: This query is conceptual and seeks to

我们可以看到，它正确识别出第一个充满特定术语的查询并选择了 keyword\\_search。

对于第二个概念性抽象的查询，正确地选择了 vector\\_search 。这种在检索漏斗开始时就进行动

7.2 通过混合搜索、关键字和语义搜索进行广泛召回

现在我们有了主管来选择策略，我们需要构建检索策略本身。我们漏斗的第一阶段是关于召回的，我们的



为此，我们将实现可以调用的三个不同的搜索功能：

- 矢量搜索：我们的标准语义搜索，但现在升级为使用元数据过滤器。
- 关键词搜索（BM25）：一种经典、强大的算法，擅长查找具有特定、精确术语的文档。
- 混合搜索：两种方法中最好的一种是使用称为倒数秩融合（RRF）的技术将向量和关键字搜索的结果结合起来。

首先，我们需要使用上一节中创建的富含元数据的块来创建一个新的高级矢量存储。

```
import numpy as np # A fundamental library for numerical operations in Python
from rank_bm25 import BM25Okapi # The library for implementing the BM25 keyword search

print("Creating advanced vector store with metadata...")
```

```
We create a new Chroma vector store, this time using our metadata-rich chunk
advanced_vector_store = Chroma.from_documents(
 documents=doc_chunks_with_metadata,
 embedding=embedding_function
)
print(f"Advanced vector store created with {advanced_vector_store._collection}")
```

这是一个简单但关键的步骤。现在， `advanced\_vector\_store` 包含与基准相同的文本，但每个

接下来，我们需要为关键词搜索做准备。BM25 算法的工作原理是分析文档中单词的频率。为了实现这-

```
print("\nBuilding BM25 index for keyword search...")

Create a list where each element is a list of words from a document
tokenized_corpus = [doc.page_content.split(" ") for doc in doc_chunks_with_metadata]

Create a list of all unique document IDs
doc_ids = [doc.metadata["id"] for doc in doc_chunks_with_metadata]

Create a mapping from a document's ID back to the full Document object for
doc_map = {doc.metadata["id"]: doc for doc in doc_chunks_with_metadata}

Initialize the BM25Okapi index with our tokenized corpus
bm25 = BM25Okapi(tokenized_corpus)
```

我们基本上是在为 BM25 索引创建必要的数据结构。`tokenized\_corpus` 是 `tokenized\_corpus`

现在我们可以定义三个检索函数。

```
Strategy 1: Pure Vector Search with Metadata Filtering
def vector_search_only(query: str, section_filter: str = None, k: int = 10):
 # This dictionary defines the metadata filter. ChromaDB will only search
 filter_dict = {"section": section_filter} if section_filter and "Unknown"
 # Perform the similarity search with the optional filter
 return advanced_vector_store.similarity_search(query, k=k, filter=filter_dict)
```

```

Strategy 2: Pure Keyword Search (BM25)
def bm25_search_only(query: str, k: int = 10):
 # Tokenize the incoming query
 tokenized_query = query.split(" ")
 # Get the BM25 scores for the query against all documents in the corpus
 bm25_scores = bm25.get_scores(tokenized_query)
 # Get the indices of the top k documents
 top_k_indices = np.argsort(bm25_scores)[::-1][:k]
 # Use our doc_map to return the full Document objects for the top results
 return [doc_map[doc_ids[i]] for i in top_k_indices]

Strategy 3: Hybrid Search with Reciprocal Rank Fusion (RRF)
def hybrid_search(query: str, section_filter: str = None, k: int = 10):
 # 1. Perform a keyword search
 bm25_docs = bm25_search_only(query, k=k)
 # 2. Perform a semantic search with the metadata filter
 semantic_docs = vector_search_only(query, section_filter=section_filter,
 # 3. Combine and re-rank the results using Reciprocal Rank Fusion (RRF)
 # Get a unique set of all documents found by either search method
 all_docs = {doc.metadata["id"]: doc for doc in bm25_docs + semantic_docs}
 # Create lists of just the document IDs from each search result
 ranked_lists = [[doc.metadata["id"] for doc in bm25_docs], [doc.metadata

 # Initialize a dictionary to store the RRF scores for each document
 rrf_scores = {}
 # Loop through each ranked list (BM25 and Semantic)
 for doc_list in ranked_lists:
 # Loop through each document ID in the list with its rank (i)
 for i, doc_id in enumerate(doc_list):
 if doc_id not in rrf_scores:
 rrf_scores[doc_id] = 0
 # The RRF formula: add 1 / (rank + k) to the score. We use k=61 ;
 rrf_scores[doc_id] += 1 / (i + 61)
 # Sort the document IDs based on their final RRF scores in descending order
 sorted_doc_ids = sorted(rrf_scores.keys(), key=lambda x: rrf_scores[x], reverse=True)
 # Return the top k Document objects based on the fused ranking
 final_docs = [doc_map[doc_id] for doc_id in sorted_doc_ids[:k]]
 return final_docs
print("\nAll retrieval strategy functions ready.")

```

我们现在已经实现了自适应检索系统的核心。

- vector\_search\_only函数是我们升级的语义搜索函数。关键新增功能是 filter=filter\_dict 参数，它允许我们从规划器的 Step 中传递 document\_section，并强制搜索仅考虑包含该元数据的块。
- bm25\_search\_only函数是我们的纯关键词检索器。它能够快速有效地查找语义搜索可能遗漏的特定术语。

- hybrid\_search函数并行运行两个搜索，然后使用 RRF 智能地合并结果。RRF 是一种简单但功能强大的算法，它根据文档在每个列表中的位置对其进行排序，从而有效地赋予在两个搜索结果中排名靠前的文档更高的权重。

让我们快速测试一下关键词搜索的效果。我们将搜索规划师确定的确切章节标题。

```
Test Keyword Search to see if it can precisely find a specific section
print("\n--- Testing Keyword Search ---")
test_query = "Item 1A. Risk Factors"
test_results = bm25_search_only(test_query)
print(f"Query: {test_query}")
print(f"Found {len(test_results)} documents. Top result section: {test_results[0]['section']}")

OUTPUT
Creating advanced vector store with metadata...
Advanced vector store created with 381 embeddings.

Building BM25 index for keyword search...
All retrieval strategy functions ready.

--- Testing Keyword Search ---
Query: Item 1A. Risk Factors
Found 10 documents. Top result section: Item 1A. Risk Factors.
```

输出结果正是我们想要的。BM25 搜索以关键词为中心，只需搜索标题，就能完美且快速地检索到 “It

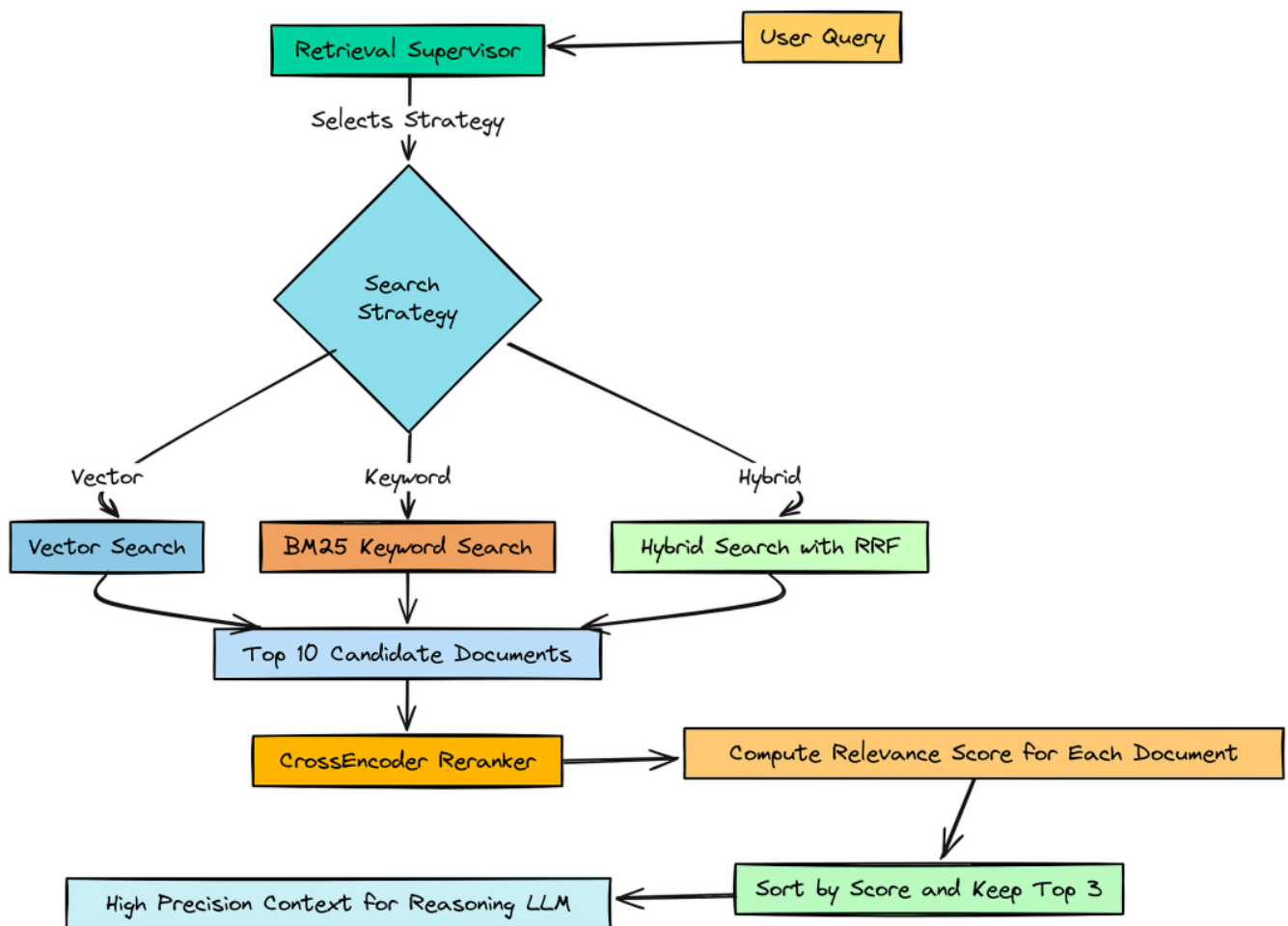
当查询包含特定关键字（例如章节标题）时，我们的主管现在可以选择这个精确的工具。

随着我们广泛的召回阶段的构建，我们拥有了一个强大的机制来查找所有可能相关的文档。然而，这张

### 7.3 使用Cross-Encoder重排序器实现高精度

我们的第一阶段检索在召回率方面做得很好，提取了 10 篇可能与我们的子问题相关的文档。

但问题是，它们只是潜在相关的。将这 10 个文档直接输入到我们的推理 LLM ，效率低下且风险很大



我们现在需要的是“精准”阶段。我们需要一种方法来检查这 10 篇候选文档，并挑选出绝对最佳的文档。

关键的区别在于这些模型的工作方式。

1. 我们的初始检索使用双编码器（嵌入模型），它分别针对查询和文档创建向量。它速度很快，非常适合搜索数百万个项目。
2. 重排序的交叉编码器将查询和单个文档作为一对，进行更深入、更细致的比较。它速度较慢，但准确率更高。

所以，我们构建一个函数来接收检索到的 10 个文档，并使用跨编码器模型为每个文档提供精确的相关性得分。然后，我们只保留前 3 个，正如我们在 config 中定义的一样。

首先，让我们初始化我们的交叉编码器模型。我们将使用 sentence-transformers 库中的一个小型但高效的模型，正如我们在配置中指定的一样。

```
from sentence_transformers import CrossEncoder # The library for using cross-encoder
print("Initializing CrossEncoder reranker...")

Initialize the CrossEncoder model using the name from our central config dict
```

```
The library will automatically download the model from the Hugging Face Hub
reranker = CrossEncoder(config["reranker_model"])
```

我们将预先训练好的重排序模型加载到内存中，这只需执行一次。我们选择的模型 `ms-marco-MiniLM`

现在我们可以创建执行重新排序的函数。

```
def rerank_documents_function(query: str, documents: List[Document]) -> List[Document]:
 # If we have no documents to rerank, return an empty list immediately.
 if not documents:
 return []

 # Create the pairs of [query, document_content] that the cross-encoder needs.
 pairs = [(query, doc.page_content) for doc in documents]

 # Use the reranker to predict a relevance score for each pair. This returns a list of scores.
 scores = reranker.predict(pairs)

 # Combine the original documents with their new scores.
 doc_scores = list(zip(documents, scores))

 # Sort the list of (document, score) tuples in descending order based on the score.
 doc_scores.sort(key=lambda x: x[1], reverse=True)

 # Extract just the Document objects from the top N sorted results.
 # The number of documents to keep is controlled by 'top_n_rerank' in our config.
 reranked_docs = [doc for doc, score in doc_scores[:config["top_n_rerank"]]]

 return reranked_docs
```

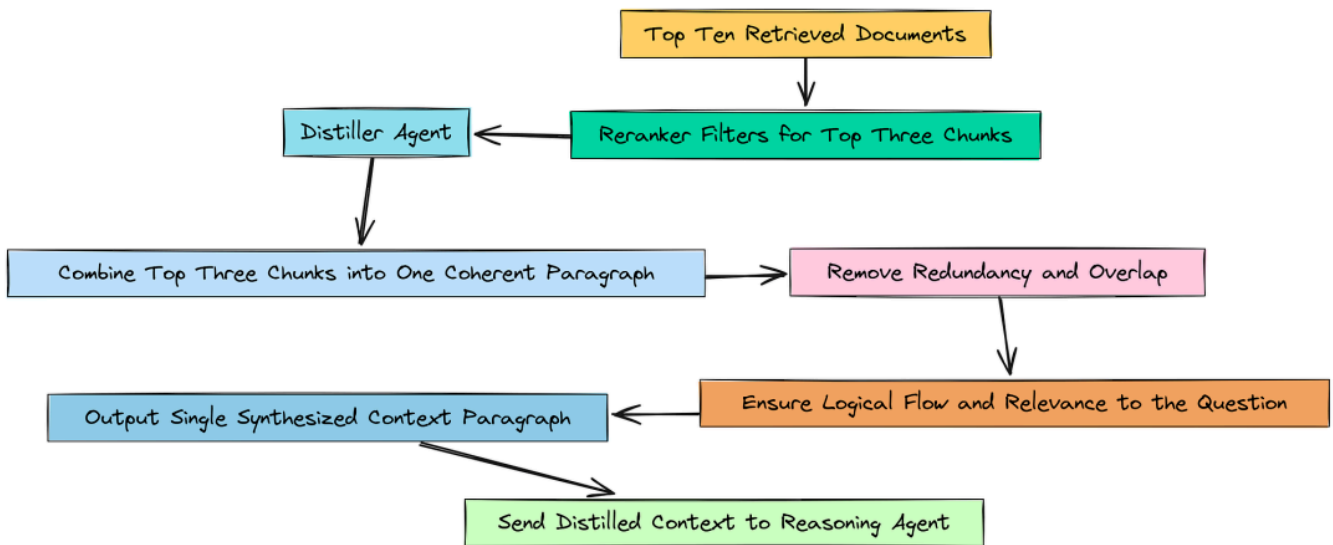
这个函数（`rerank_documents_function`）是我们精确度阶段的主要部分。它接收 `query` 和来自

在这里，模型不会创建嵌入，而是对查询与每个文档内容进行全面比较，并为每个文档生成相关性分数。

这种漏斗式方法，从高召回率的第一阶段过渡到高精度的第二阶段，是生产级 RAG 系统的一个组成部分。

## 7.4 使用上下文提炼进行合成

现在，我们的检索漏斗运行得非常好。我们先进行大范围搜索，找到了 10 篇可能相关的文档。然后



虽然它们都相关，但可能包含冗余信息或重叠句子。将它们呈现为三个不同的区块，对于LLM来说，处理

我们检索漏斗的最后一个阶段是语境提炼。目标很简单：提取最相关的三个文档块，并将它们提炼成一

这个提炼步骤充当了最后的压缩层。它确保输入到我们更昂贵的推理代理中的上下文尽可能密集且信息

首先，我们需要设计引导其行为的提示。

```
The prompt for our distiller agent, instructing it to synthesize and be coherent
distiller_prompt = ChatPromptTemplate.from_messages([
 ("system", """"You are a helpful assistant. Your task is to synthesize the
The goal is to provide a clear and coherent context that directly answers the
Focus on removing redundant information and organizing the content logically.
 ("human", "Retrieved Documents:\n{context}") # The content of our top 3 documents
])
```

我们给这个代理一个非常专注的任务，告诉它：“这里有一些文本片段。你唯一的任务就是把它们合并成

现在，我们可以组装简单的 `distiller\_agent` 。

```
Create the agent by piping our prompt to the reasoning LLM and a string output
distiller_agent = distiller_prompt | reasoning_llm | StrOutputParser()
print("Contextual Distiller Agent created.")
```

这是另一个简单的 LCEL 链。我们获取 `distiller_prompt`，将其通过管道传输到强大的 `reasoning_llm`，最后通过 `StrOutputParser` 输出字符串。

创建好这个 `distiller_agent` 后，我们的多阶段检索漏斗就完成了。在我们的主代理循环中，每个子问题都会经过这个漏斗。

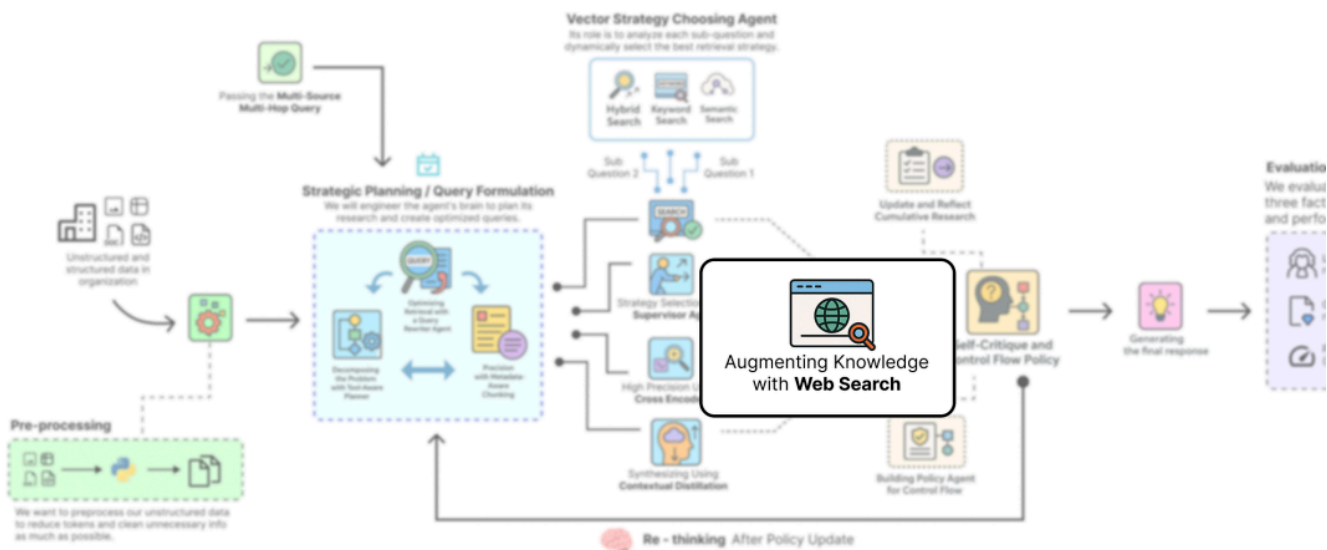
1. 主管：选择一种检索策略（`vector`、`keyword` 或 `hybrid`）。
2. 召回阶段：执行选定的策略以获取前 10 个文档。
3. 精确阶段：使用 `rerank_documents_function` 获取前 3 个文档。
4. 蒸馏阶段：使用 `distiller_agent` 将前 3 个文档压缩成一个干净的段落。

这个多阶段流程确保我们的agent所处理的证据具有最高的质量。下一步是赋予我们的agent超越其内部知识的能力，并搜索网络。

### 八、利用网络搜索增强知识

现在，我们的检索漏斗现在非常强大，但它有一个巨大的盲点，它只能看到2023 年的 10-K 文件的内部数据。

要真正打造一个“深度思考”的智能体，它需要能够认识到自身知识的局限性，并在其他地方寻找答案。





在这一步，我们用一个新工具 —Web Search— 来增强我们agent的功能。这将使我们的系统从一个

为此，我们将使用 Tavily Search API 。这是一个专为 LLM 打造的搜索引擎，提供简洁、无广告

所以，我们需要做的第一件事就是初始化 Tavily 搜索工具本身。

```
from langchain_community.tools.tavily_search import TavilySearchResults

Initialize the Tavily search tool.
k=3: This parameter instructs the tool to return the top 3 most relevant search results.
web_search_tool = TavilySearchResults(k=3)
```

创建一个 Tavily 搜索工具的实例，供我们的agent调用。k=3 参数是一个很好的起点，它提供了一!

现在，原始的 API 响应并非我们真正需要的。我们的下游组件，重新排序器和提取器，都设计为与特定

```
def web_search_function(query: str) -> List[Document]:
 # Invoke the Tavily search tool with the provided query.
 results = web_search_tool.invoke({"query": query})

 # Format the results into a list of LangChain Document objects.
 # We use a list comprehension for a concise and readable implementation.
 return [
 Document(
 # The main content of the search result goes into 'page_content'.
 page_content=res["content"],
 # We store the source URL in the 'metadata' dictionary for citation.
 metadata={"source": res["url"]}
) for res in results
]
```

这个 web\\_search\\_function 充当着一个至关重要的适配器，它调用 web\\_search\\_tool.in

然后，使用列表推导循环遍历这些结果并将它们整齐地重新打包到我们的管道所需的 Document 对象中

page\\_content 获取主要文本，重要的是，我们将 url 存储在 metadata 中，这确保了当我们的

这样，我们的外部知识源看起来和感觉起来与我们的内部知识源完全一样，从而允许我们对两者使用相

函数准备就绪后，我们来快速测试一下，确保它能按预期工作。我们将使用与主要挑战第二部分相关的

```
Test the web search function with a query about AMD's 2024 strategy
print("\n--- Testing Web Search Tool ---")
test_query_web = "AMD AI chip strategy 2024"
test_results_web = web_search_function(test_query_web)
print(f"Found {len(test_results_web)} results for query: '{test_query_web}'")
Print a snippet from the first result to see what we got back
if test_results_web:
 print(f"Top result snippet: {test_results_web[0].page_content[:250]}...")
```

#### OUTPUT ####

Web search tool (Tavily) initialized.

--- Testing Web Search Tool ---

Found 3 results for query: 'AMD AI chip strategy 2024'

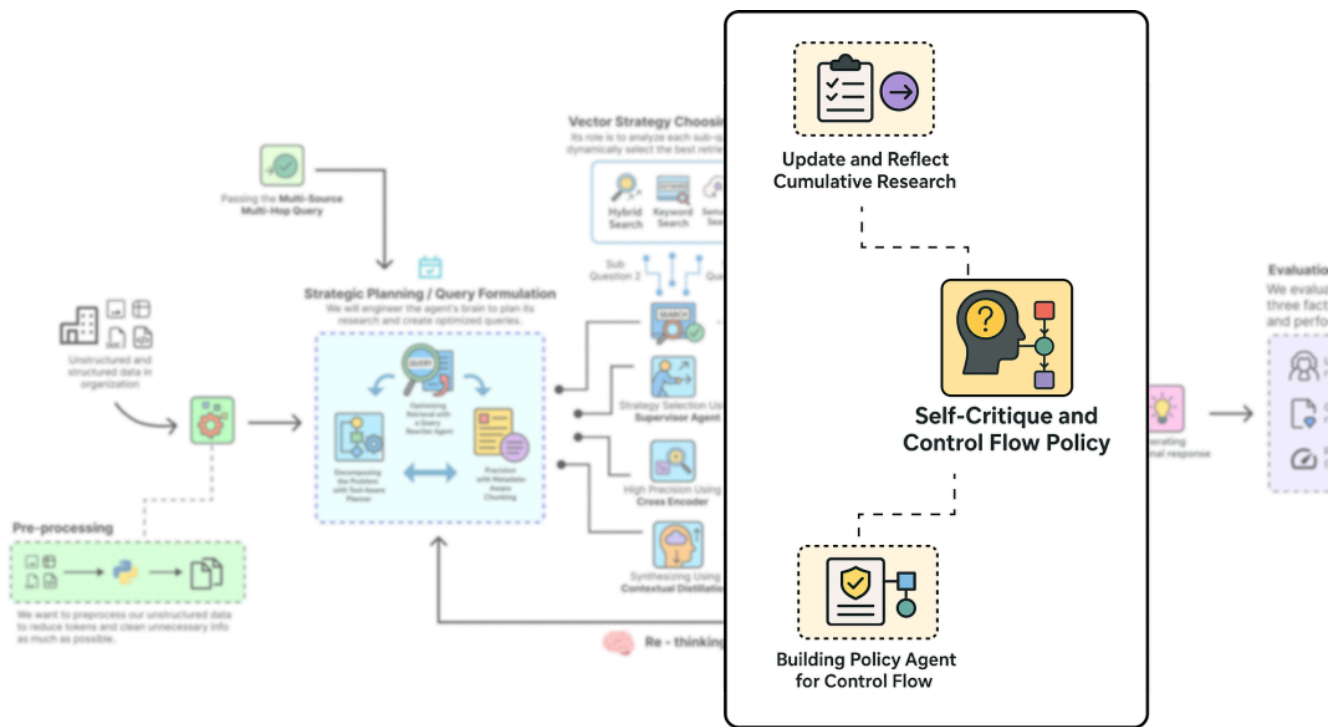
Top result snippet: AMD has intensified its battle with Nvidia in the AI chip

输出结果证实了我们的工具运行正常。它找到了 3 个与我们的查询相关的网页。顶部结果中的片段正是

我们的智能体现在拥有了一扇通往外部世界的窗户，它的规划器可以智能地决定何时打开这扇窗户。最后

## 九、自我批评和控制流政策

到目前为止，我们已经构建了一台强大的研究机器。我们的agent可以制定计划，选择合适的工具，并



这就是我们构建agent自主认知核心的地方。在每个研究步骤之后，我们的agent都会暂停并进行反思。

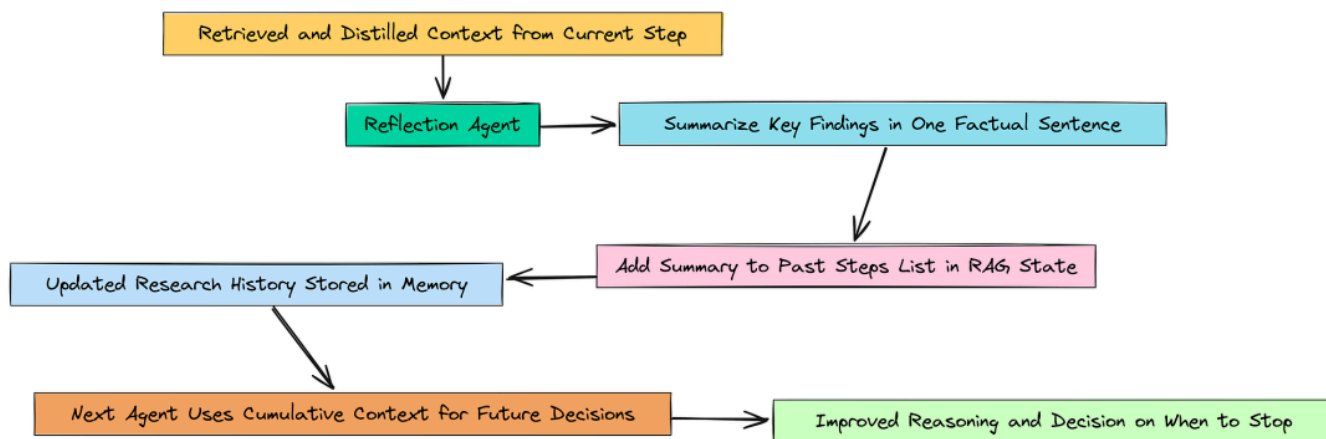
这种自我批评循环使我们的系统从脚本化的工作流程提升为自主的代理。正是这种机制让它能够判断何

我们将使用两个新的专门代理来实现这一点：

- Reflection Agent：该代理将从已完成的步骤中提取上下文，并创建一个简洁的单句摘要，该摘要随后会被添加到我们代理的“研究历史”中。
- Policy Agent：这是战略大师。经过反思，它将根据原计划审查整个研究历史，并做出一个关键决定：CONTINUE\_PLAN 还是 FINISH。

## 9.1 更新并反映累积研究历史

当我们的代理完成一个研究步骤（例如，检索并提取有关 NVIDIA 风险的信息）后，我们不想就此打



我们将构建一个反射代理，它将从当前步骤中提取丰富、精炼的上下文，并将其总结成一个单一、真实的

首先，让我们为该代理创建提示。

```

The prompt for our reflection agent, instructing it to be concise and factual
reflection_prompt = ChatPromptTemplate.from_messages([
 ("system", """"You are a research assistant. Based on the retrieved context, summarize key findings in one factual sentence. This summary will be added to our research history. Be factual and to the point."""),
 ("human", "Current sub-question: {sub_question}\n\nDistilled context:\n{context}")
])

```

我们让这个智能体像一个勤奋的研究助理一样工作。它的任务不是发挥创造力，而是做好笔记。它会阅读

```

Create the agent by piping our prompt to the reasoning LLM and a string output parser
reflection_agent = reflection_prompt | reasoning_llm | StrOutputParser()
print("Reflection Agent created.")

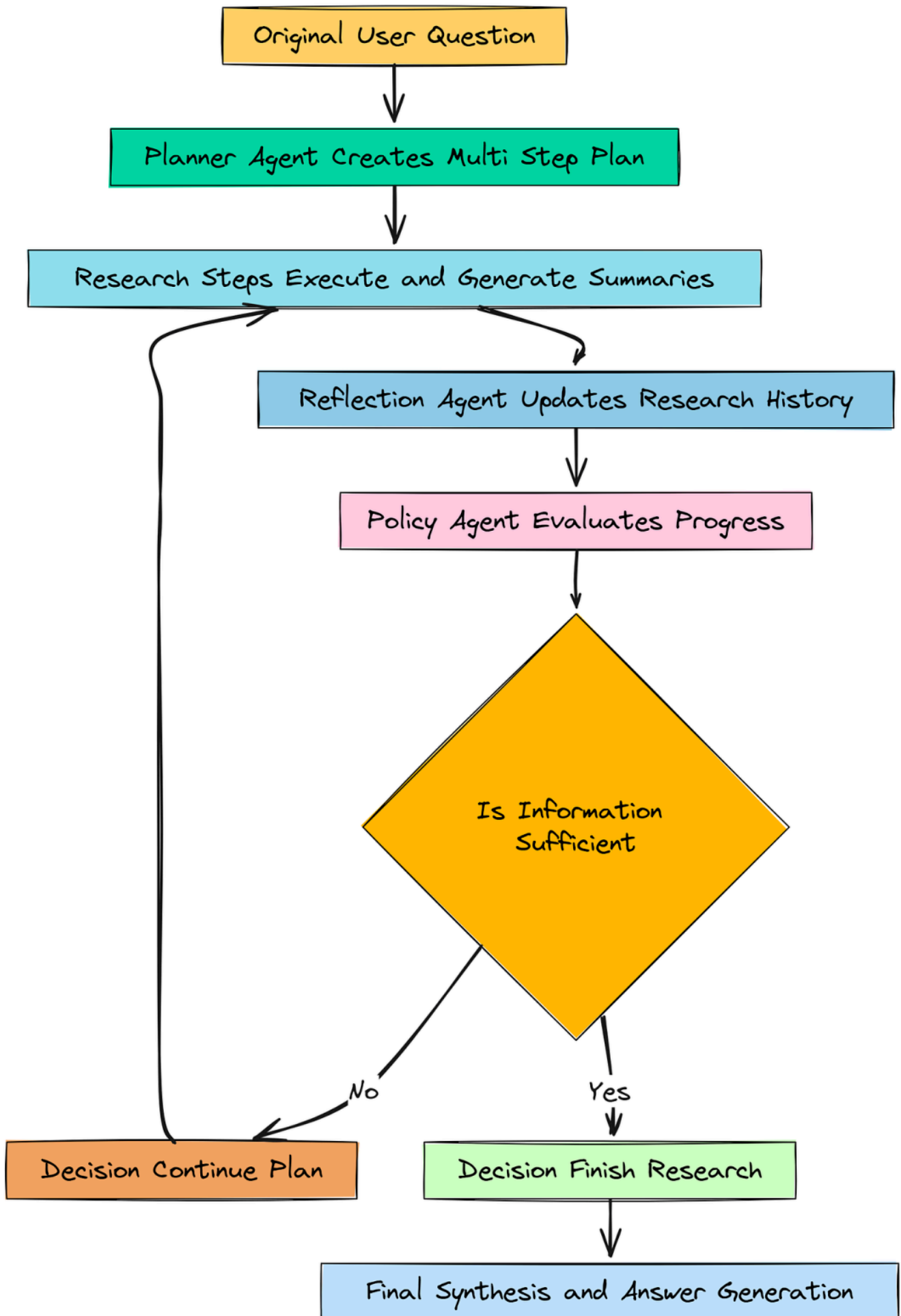
```

这个 `reflection_agent` 是我们认知循环的一部分。通过创建这些简洁的摘要，它构建了一个清晰

## 9.2 构建控制流策略代理

这是我们自主agent的大脑。在 `reflection\_agent` 更新了研究历史记录后，策略代理便开始发

它的工作是查看代理所知道的一切——原始问题、初步计划以及已完成步骤的完整摘要历史记录，并做出i



我们将首先使用 Pydantic 模型定义其决策的结构。

```
class Decision(BaseModel):
 # The decision must be one of these two actions.
 next_action: Literal["CONTINUE_PLAN", "FINISH"]
 # The agent must justify its decision.
 justification: str
```

这个 Decision 类强制我们的策略代理做出明确的二元选择，并解释其推理，这使得它的行为透明且易于理解。

接下来，我们设计指导其决策过程的提示。

```
The prompt for our policy agent, instructing it to act as a master strategist.
policy_prompt = ChatPromptTemplate.from_messages([
 ("system", """"You are a master strategist. Your role is to analyze the research history and decide whether to continue the plan or finish it. You have the original question, the initial plan, and a log of completed steps. - If the collected information in the Research History is sufficient to complete the plan, decide to FINISH. - Otherwise, if the plan is not yet complete, decide to CONTINUE_PLAN."""),
 ("human", "Original Question: {question}\n\nInitial Plan:\n{plan}\n\nResearch History:\n{research_history}\n\n")
)
```

我们要求LLM进行元分析，它不是回答问题本身，而是对研究过程的状态进行推理。它将现有信息（包括初始计划、研究历史、已完成步骤）与初始计划进行比较，并决定是继续计划还是完成计划。

现在，我们可以组装 policy\_agent 。

```
Create the agent by piping our prompt to the reasoning LLM and structuring the output.
policy_agent = policy_prompt | reasoning_llm.with_structured_output(Decision)
print("Policy Agent created.")

Now, let's test the policy agent with two different states of our research history.
print("\n--- Testing Policy Agent (Incomplete State) ---")
```

```

First, a state where only Step 1 is complete.
plan_str = json.dumps([s.dict() for s in test_plan.steps])
incomplete_history = "Step 1 Summary: NVIDIA's 10-K states that the semiconducto
decision1 = policy_agent.invoke({"question": complex_query_adv, "plan": plan_
print(f"Decision: {decision1.next_action}, Justification: {decision1.justific
print("\n--- Testing Policy Agent (Complete State) ---")

Second, a state where both Step 1 and Step 2 are complete.
complete_history = incomplete_history + "\nStep 2 Summary: In 2024, AMD launc
decision2 = policy_agent.invoke({"question": complex_query_adv, "plan": plan_
print(f"Decision: {decision2.next_action}, Justification: {decision2.justific

```

为了正确测试我们的 `policy\_agent`，我们模拟了代理生命周期中的两个不同时刻。在第一个测试

让我们来研究一下它在每种情况下做出的决定。

```

OUTPUT
Policy Agent created.

--- Testing Policy Agent (Incomplete State) ---
Decision: CONTINUE_PLAN, Justification: The research has only identified NVII

--- Testing Policy Agent (Complete State) ---
Decision: FINISH, Justification: The research history now contains comprehens

```

让我们了解一下输出.....

在不完整状态下，`agent`正确识别出缺少有关 `AMD` 策略的信息。它查看了计划，发现下一步是使用网

在完整阶段，在获得网络搜索的摘要后，它再次分析了其历史记录。这一次，它意识到自己已经掌握

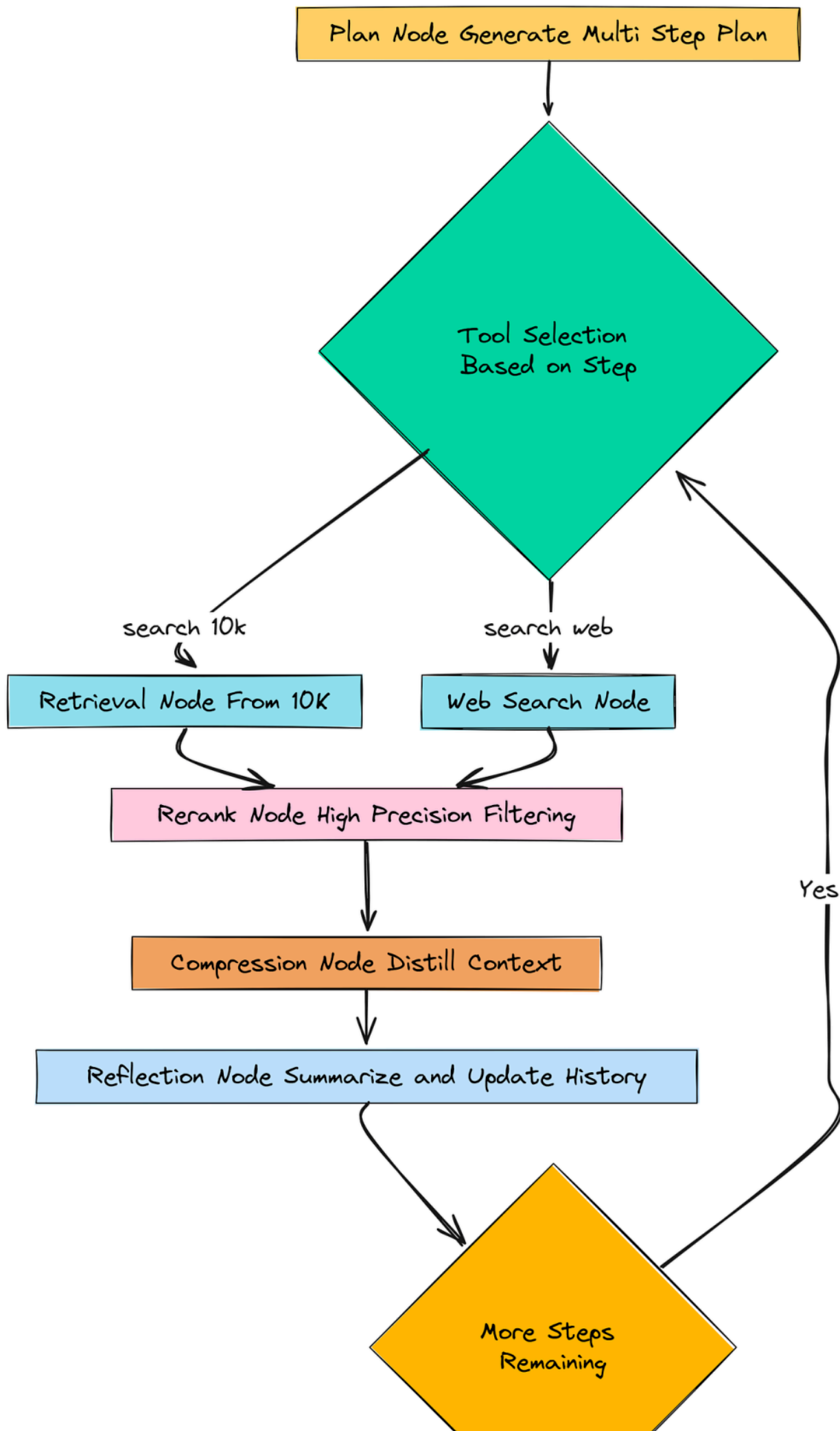
通过这个 `policy\_agent`，我们构建了自主系统的大脑。最后一步是使用 `LangGraph` 将所有这些

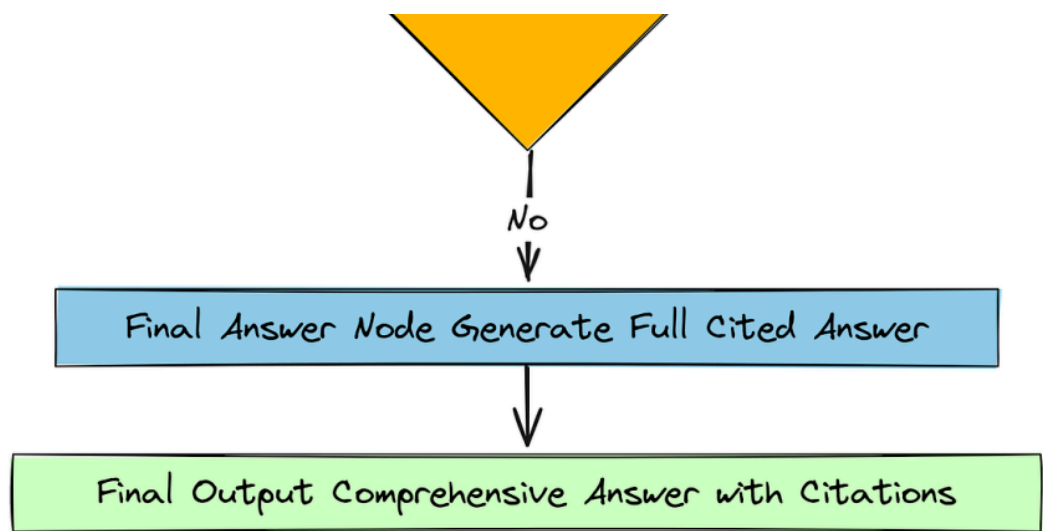
## 十、定义图形节点

我们已经设计了所有这些酷炫的专业`agent`。现在是时候将它们转化为我们工作流程的实际构建块了。在



我们将为agent需要采取每个主要步骤创建一个节点。





首先，我们需要一个简单的辅助函数。由于我们的agent经常需要查看研究历史记录，因此我们需要一种

```

A helper function to format the research history for prompts
def get_past_context_str(past_steps: List[PastStep]) -> str:
 # This takes the list of PastStep dictionaries and joins them into a single string
 # Each step is clearly labeled for the LLM to understand the context.
 return "\\n\\n".join([f"Step {s['step_index']}: {s['sub_question']}" for s in past_steps])

```

我们创建一个实用程序，它将在我们的几个节点内使用，为我们的提示提供历史背景。

现在来看看我们的第一个真实节点： `plan\_node` 。这是我们agent推理的起点。它唯一的任务就是

```

Node 1: The Planner
def plan_node(state: RAGState) -> Dict:
 console.print("--- 🧠: Generating Plan ---")
 # We call the planner_agent we created earlier, passing in the user's original question
 plan = planner_agent.invoke({"question": state["original_question"]})
 pprint(plan)
 # We return a dictionary with the updates for our RAGState.
 # LangGraph will automatically merge this into the main state.
 return {"plan": plan, "current_step_index": 0, "past_steps": []}

```

此节点启动所有操作。它从状态中获取 `original\_question`，获取 `plan`，然后将 `current\_s`

接下来，我们需要实际查找信息的节点。由于我们的规划器可以在两种工具之间进行选择，因此我们需

```
Node 2a: Retrieval from the 10-K document
def retrieval_node(state: RAGState) -> Dict:
 # First, get the details for the current step in the plan.
 current_step_index = state["current_step_index"]
 current_step = state["plan"].steps[current_step_index]
 console.print(f"--- 🔍: Retrieving from 10-K (Step {current_step_index} +

 # Use our query rewriter to optimize the sub-question for search.
 past_context = get_past_context_str(state['past_steps'])
 rewritten_query = query_rewriter_agent.invoke({
 "sub_question": current_step.sub_question,
 "keywords": current_step.keywords,
 "past_context": past_context
 })
 console.print(f" Rewritten Query: {rewritten_query}")

 # Get the supervisor's decision on which retrieval strategy is best.
 retrieval_decision = retrieval_supervisor_agent.invoke({"sub_question":
 console.print(f" Supervisor Decision: Use `{retrieval_decision.strategy}`

 # Based on the decision, execute the correct retrieval function.
 if retrieval_decision.strategy == 'vector_search':
 retrieved_docs = vector_search_only(rewritten_query, section_filter=
 elif retrieval_decision.strategy == 'keyword_search':
 retrieved_docs = bm25_search_only(rewritten_query, k=config['top_k_re
 else: # hybrid_search
 retrieved_docs = hybrid_search(rewritten_query, section_filter=curre

 # Return the retrieved documents to be added to the state.
 return {"retrieved_docs": retrieved_docs}
```

这个节点可以执行大量智能工作，不仅仅是一个简单的检索器。它编排了一个微型管道：它重写查询，

现在，我们需要另一个工具的对应节点：网络搜索。

```

Node 2b: Retrieval from the Web
def web_search_node(state: RAGState) -> Dict:
 # Get the details for the current step.
 current_step_index = state["current_step_index"]
 current_step = state["plan"].steps[current_step_index]
 console.print(f"--- 🌐: Searching Web (Step {current_step_index + 1}): {current_step.sub_question}")

 # Rewrite the sub-question for a web search engine.
 past_context = get_past_context_str(state['past_steps'])
 rewritten_query = query_rewriter_agent.invoke({
 "sub_question": current_step.sub_question,
 "keywords": current_step.keywords,
 "past_context": past_context
 })
 console.print(f" Rewritten Query: {rewritten_query}")
 # Call our web search function.
 retrieved_docs = web_search_function(rewritten_query)
 # Return the results.
 return {"retrieved_docs": retrieved_docs}

```

这个 `web\_search\_node` 更简单，因为它不需要 supervisor，只有一种搜索网络的方法。但它仍

检索文档（无论来自哪个来源）后，我们需要运行精度和综合漏斗。我们将为每个阶段创建一个节点。目

```

Node 3: The Reranker
def rerank_node(state: RAGState) -> Dict:
 console.print("--- 🎯: Reranking Documents ---")
 # Get the current step's details.
 current_step_index = state["current_step_index"]
 current_step = state["plan"].steps[current_step_index]
 # Call our reranking function on the documents we just retrieved.
 reranked_docs = rerank_documents_function(current_step.sub_question, state["retrieved_docs"])
 console.print(f" Reranked to top {len(reranked_docs)} documents.")
 # Update the state with the high-precision documents.
 return {"reranked_docs": reranked_docs}

```

该节点获取 `retrieved\_docs`（我们对 10 个文档的广泛回忆）并使用交叉编码器将它们过滤到前

接下来， `compression\_node` 将获取前 3 个文档并对其进行提炼。

```
Node 4: The Compressor / Distiller
def compression_node(state: RAGState) -> Dict:
 console.print("--- ✂ : Distilling Context ---")
 # Get the current step's details.
 current_step_index = state["current_step_index"]
 current_step = state["plan"].steps[current_step_index]
 # Format the top 3 documents into a single string.
 context = format_docs(state["reranked_docs"])
 # Call our distiller agent to synthesize them into one paragraph.
 synthesized_context = distiller_agent.invoke({"question": current_step.sub_question})
 console.print(f" Distilled Context Snippet: {synthesized_context[:200]}")
 # Update the state with the final, clean context.
 return {"synthesized_context": synthesized_context}
```

此节点是我们检索漏斗的最后一步。它获取 `reranked\_docs` 并生成一个干净的 `synthesized\_context`。

现在我们有证据，我们需要反思它并更新我们的研究历史。这就是 `reflection\_node` 的工作。

```
Node 5: The Reflection / Update Step
def reflection_node(state: RAGState) -> Dict:
 console.print("--- : Reflecting on Findings ---")
 # Get the current step's details.
 current_step_index = state["current_step_index"]
 current_step = state["plan"].steps[current_step_index]
 # Call our reflection agent to summarize the findings.
 summary = reflection_agent.invoke({"sub_question": current_step.sub_question})
 console.print(f" Summary: {summary}")

 # Create a new PastStep dictionary with all the results from this step.
 new_past_step = {
 "step_index": current_step_index + 1,
 "sub_question": current_step.sub_question,
 "retrieved_docs": state['reranked_docs'], # We save the reranked docs
 "summary": summary
 }
 # Append the new step to our history and increment the step index to move forward.
 return {"past_steps": state["past_steps"] + [new_past_step], "current_step_index": current_step_index + 1}
```

这个节点是我们agent的簿记员。它调用 `reflection\_agent` 创建摘要，然后将当前研究周期的所

最后，当研究完成时，我们需要最后一个节点来生成最终答案。

```
Node 6: The Final Answer Generator
def final_answer_node(state: RAGState) -> Dict:
 console.print("--- ✅: Generating Final Answer with Citations ---")
 # First, we need to gather all the evidence we've collected from ALL past steps
 final_context = ""
 for i, step in enumerate(state['past_steps']):
 final_context += f"\n--- Findings from Research Step {i+1} ---\n"
 # We include the source metadata (section or URL) for each document found
 for doc in step['retrieved_docs']:
 source = doc.metadata.get('section') or doc.metadata.get('source')
 final_context += f"Source: {source}\nContent: {doc.page_content}\n"

 # We create a new prompt specifically for generating the final, citable answer
 final_answer_prompt = ChatPromptTemplate.from_messages([
 ("system", """"You are an expert financial analyst. Synthesize the research findings into a clear, concise final answer. Your answer must be grounded in the provided context. At the end of any sentence, include citations for the sources used."""),
 ("human", "Original Question: {question}\n\nResearch History and Context: {context}")
])

 # We create a temporary agent for this final task and invoke it.
 final_answer_agent = final_answer_prompt | reasoning_llm | StrOutputParser
 final_answer = final_answer_agent.invoke({"question": state['original_question'], "context": final_context})
 # Update the state with the final answer.
 return {"final_answer": final_answer}
```

这个 `final\_answer\_node` 是我们最终的收官之作。它将 `past\_steps` 历史记录中每一步的所

定义好所有节点后，我们现在有了agent的所有构建块。下一步是定义连接它们并控制图的流程的“线”

## 十一、定义条件边

至此，我们已经构建好了所有节点。我们有一个规划器、检索器、一个重排序器、一个提取器和一个反思

这就是 LangGraph 中边的作用。简单的边很简单，“在节点 A 之后，总是指向节点 B”。但真正

条件边是一种函数，它查看代理的当前内存（ RAGState ）并做出决策，根据情况将工作流程路由到

我们的agent需要两个关键的决策功能：

- 工具路由器（ route\_by\_tool ）： 制定计划后，此功能将查看计划的当前步骤并决定是否将工作流发送到 retrieve\_10k 节点或 retrieve\_web 节点。
- 主控制循环（ should\_continue\_node ）： 这是最重要的一个。在每个研究步骤完成并反思之后，此函数将调用我们的 policy\_agent 来决定是继续执行计划的下一步，还是完成研究并生成最终答案。

首先，让我们构建简单的工具路由器。

```
Conditional Edge 1: The Tool Router
def route_by_tool(state: RAGState) -> str:
 # Get the index of the current step we are on.
 current_step_index = state["current_step_index"]
 # Get the full details of the current step from the plan.
 current_step = state["plan"].steps[current_step_index]
 # Return the name of the tool specified for this step.
 # LangGraph will use this string to decide which node to go to next.
 return current_step.tool
```

这个函数非常简单，但至关重要。它就像一个总机，从状态中读取 `current\_step\_index`，在 p

现在我们需要创建一个函数来控制代理的主要推理循环。这就是我们的 `policy\_agent` 发挥作用的

```
Conditional Edge 2: The Main Control Loop
def should_continue_node(state: RAGState) -> str:
 console.print("--- 🧠 : Evaluating Policy ---")
 # Get the index of the step we are about to start.
 current_step_index = state["current_step_index"]

 # First, check our basic stopping conditions.
 # Condition 1: Have we completed all the steps in the plan?
 if current_step_index >= len(state["plan"].steps):
 console.print(" -> Plan complete. Finishing.")
 return "finish"
```



```

Condition 2: Have we exceeded our safety limit for the number of iterations?
if current_step_index >= config["max_reasoning_iterations"]:
 console.print(" -> Max iterations reached. Finishing.")
 return "finish"

A special case: If the last retrieval step failed to find any documents,
there's no point in reflecting. It's better to just move on to the next step.
if state.get("reranked_docs") is not None and not state["reranked_docs"]:
 console.print(" -> Retrieval failed for the last step. Continuing with the next step.")
 return "continue"

If none of the basic conditions are met, it's time to ask our Policy Agent for a decision.
We format the history and plan into strings for the prompt.
history = get_past_context_str(state['past_steps'])
plan_str = json.dumps([s.dict() for s in state['plan'].steps])

Invoke the policy agent to get its strategic decision.
decision = policy_agent.invoke({"question": state["original_question"], "history": history, "plan": plan_str})
console.print(f" -> Decision: {decision.next_action} | Justification: {decision.justification}")

Based on the agent's decision, return the appropriate signal.
if decision.next_action == "FINISH":
 return "finish"
else: # CONTINUE_PLAN
 return "continue"

```

这个 `should_continue_node` 函数是我们代理控制流的认知核心。它在每个 `reflection_node` 调用后执行。

它首先检查简单的、硬编码的停止条件。计划是否用完了所有步骤？是否达到了 `max_reasoning_steps`？

如果这些检查通过，它就会调用我们强大的 `policy_agent`。它为策略代理提供所需的所有上下文：

最后，它获取 `policy_agent` 的结构化输出（`CONTINUE_PLAN` 或 `FINISH`），并返回简单字符串信号。

现在，我们已经定义了节点（专家）和条件边（对话规则），并且拥有了所需的一切。

现在是时候将所有这些部分组装成一个完整的、可运行的 `StateGraph` 了。

## 十二、连接深度思考 RAG 机器

我们已经准备好所有单独的组件：

- 我们的节点（工作者）

- 我们的条件边缘（经理）。

现在是时候将它们全部连接起来形成一个单一的、有凝聚力的系统了。

我们将使用 LangGraph 的 StateGraph 来定义智能体的完整认知架构。在这里，我们规划了agent

我们要做的第一件事是创建一个 StateGraph 实例。我们会告诉它，它将要传递的“状态”是我们的

```
from langgraph.graph import StateGraph, END # Import the main graph component

Instantiate the graph, telling it to use our RAGState TypedDict as its state
graph = StateGraph(RAGState)
```

现在我们有了一个空图。下一步是添加我们之前定义的所有节点。 `.add_node()` 方法接受两个参数

```
Add all of our Python functions as nodes in the graph
graph.add_node("plan", plan_node) # The node that creates the plan
graph.add_node("retrieve_10k", retrieval_node) # The node for internal retrieval
graph.add_node("retrieve_web", web_search_node) # The node for external retrieval
graph.add_node("rerank", rerank_node) # The node that performs reranking
graph.add_node("compress", compression_node) # The node that distills information
graph.add_node("reflect", reflection_node) # The node that summarizes the process
graph.add_node("generate_final_answer", final_answer_node) # The node that synthesizes the final answer
```

现在所有专家都到齐了。最后也是最关键的一步是定义连接他们的“线路”。在这里，我们使用 `.add_conditional_edges()`

```
The entry point of our graph is the "plan" node. Every run starts here.
graph.set_entry_point("plan")

After the "plan" node, we use our first conditional edge to decide which tool to use
graph.add_conditional_edges(
 "plan", # The source node
 route_by_tool, # The function that makes the decision
```

```

 {
 # A dictionary mapping the function's output string to
 "search_10k": "retrieve_10k",
 "search_web": "retrieve_web",
 },
)

After retrieving from either the 10-K or the web, the flow is linear for a
graph.add_edge("retrieve_10k", "rerank") # After internal retrieval, always (
graph.add_edge("retrieve_web", "rerank") # After web retrieval, also always (
graph.add_edge("rerank", "compress") # After reranking, always go to co
graph.add_edge("compress", "reflect") # After compressing, always go to r

After the "reflect" node, we hit our main conditional edge, which controls
graph.add_conditional_edges(
 "reflect", # The source node
 should_continue_node, # The function that calls our Policy Agent
 {
 # A dictionary mapping the decision to the next step
 "continue": "plan", # If the decision is "continue", we loop back to
 "finish": "generate_final_answer", # If the decision is "finish", we
 },
)

The "generate_final_answer" node is the last step before the end.
graph.add_edge("generate_final_answer", END) # After generating the answer, t
print("StateGraph constructed successfully.")

```

这是我们的agent大脑的蓝图。让我们来追踪一下它的流程：

1. 一切总是按 plan 开始。
2. 然后， route\_by\_tool 条件边充当开关，将流引导至 retrieve\_10k 或 retrieve\_web 。
3. 无论运行哪个检索器，输出总是通过 rerank -> compress -> reflect 管道进行传输。
4. 这引出了最重要的部分： should\_continue\_node 条件边。这是我们循环推理的核心。
5. 如果策略代理说 CONTINUE\_PLAN ， 边缘节点会将工作流一路发送回 plan 节点。我们返回到 plan （而不是直接发送到下一个检索器），以便 route\_by\_tool 能够正确地路由计划中的下一步 。
6. 如果策略代理说 FINISH ， 边缘将中断循环并将工作流发送到 generate\_final\_answer 节点。
7. 最后，在生成答案后，图表在 END 处终止。

我们已经成功定义了深度思考agent的完整、复杂且循环的架构。剩下要做的就是将这个蓝图编译成一个可运行的应用程序，并将其可视化，以查看我们构建的内容。

### 十三、编译和可视化迭代 workflow

当我们的图完全连接好后，组装过程的最后一步就是编译它。`.compile` `.compile()` 方法将我们对

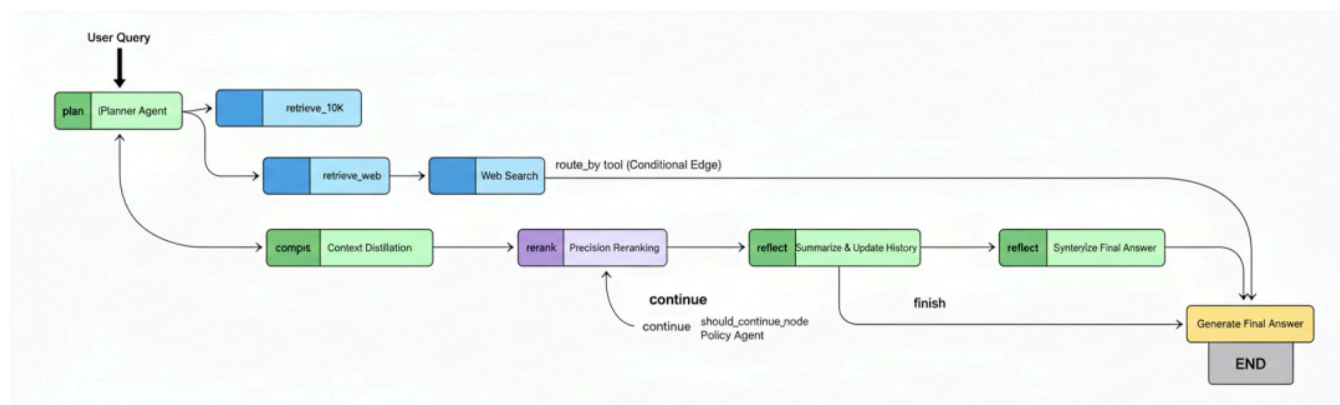
然后，我们可以使用内置的 `LangGraph` 实用程序生成图的示意图。可视化 workflow 对于理解和调试：

所以，基本上，我们正在将我们的蓝图变成一台真正的机器。

```
The .compile() method takes our graph definition and creates a runnable object
deep_thinking_rag_graph = graph.compile()
print("Graph compiled successfully.")

Now, let's visualize the architecture we've built.
try:
 from IPython.display import Image, display
 # We can get a PNG image of the graph's structure.
 png_image = deep_thinking_rag_graph.get_graph().draw_png()
 # Display the image in our notebook.
 display(Image(png_image))
except Exception as e:
 # This can fail if pygraphviz and its system dependencies are not installed
 print(f"Graph visualization failed: {e}. Please ensure pygraphviz is installed")
```

`deep\_thinking\_rag\_graph` 对象现在已经成为我们功能齐全的agent了。可视化代码随后调用



我们可以清晰的看到：

- 初始分支逻辑是 `route_by_tool` 在 `retrieve_10k` 和 `retrieve_web` 之间进行选择。

- 每个研究步骤的线性处理管道（rerank - compress - reflect）。
- 关键的反馈循环，其中 should\_continue 边缘将工作流发送回 plan 节点以开始下一个研究周期。
- 研究完成后，最后的“出口坡道”将通向 generate\_final\_answer。

这就是一个能够思考的系统的架构。现在，让我们来测试一下。

## 十四、运行深度思考管道

我们已经设计出一个推理引擎。现在是时候看看它能否在我们基线系统彻底失败的地方取得成功。

我们将使用完全相同的多跳、多源挑战查询来调用已编译的 `deep\_thinking\_rag\_graph`。我们

这是本节的计划：

- 调用图表：我们将运行我们的代理并观察它如何执行其计划、在工具之间切换以及构建其研究历史。
- 分析最终输出：我们将检查最终的综合答案，看看它是否成功整合了来自 10-K 和网络的信息。
- 比较结果：我们将进行最后的并列比较，以明确突出我们的深度思考代理的架构优势。

我们将设置初始输入，它只是一个包含 `original_question` 的字典，然后调用 `stream.stream()` 方法。`stream` 方法非常适合调试和观察，因为它会在每个节点完成工作后返回图的状态。

```
This will hold the final state of the graph after the run is complete.
final_state = None
The initial input for our graph, containing the original user query.
graph_input = {"original_question": complex_query_adv}

print("---- Invoking Deep Thinking RAG Graph ----")
We use .stream() to watch the agent's process in real-time.
"values" mode means we get the full RAGState object after each step.
for chunk in deep_thinking_rag_graph.stream(graph_input, stream_mode="values"):
 # The final chunk in the stream will be the terminal state of the graph.
 final_state = chunk
print("\n--- Graph Stream Finished ----")
```

这个循环就是我们的agent开始运作的地方。每次迭代时，LangGraph 都会执行 workflow 中的下一个节点。

#### OUTPUT ####

--- Invoking Deep Thinking RAG Graph ---

--- 🧠: Generating Plan ---

plan:

steps:

- sub\_question: What are the key risks related to competition as stated in tool: search\_10k  
...
- sub\_question: What are the recent news and developments in AMD's AI chip tool: search\_web  
...

--- 🔍: Retrieving from 10-K (Step 1: ...) ---

Rewritten Query: key competitive risks for NVIDIA in the semiconductor industry  
Supervisor Decision: Use `hybrid\_search`. ...

--- 🎯: Reranking Documents ---

Reranked to top 3 documents.

--- ✂️: Distilling Context ---

Distilled Context Snippet: NVIDIA operates in the intensely competitive semiconductor market.

--- 🤔: Reflecting on Findings ---

Summary: According to its 2023 10-K, NVIDIA operates in an intensely competitive market.

--- 🚦: Evaluating Policy ---

-> Decision: CONTINUE\_PLAN | Justification: The first step...has been completed.

--- 🌐: Searching Web (Step 2: ...) ---

Rewritten Query: AMD AI chip strategy news and developments 2024...

--- 🎯: Reranking Documents ---

Reranked to top 3 documents.

--- ✂️: Distilling Context ---

Distilled Context Snippet: AMD has ramped up its challenge to Nvidia in the AI chip market.

--- 🤔: Reflecting on Findings ---

Summary: In 2024, AMD is aggressively competing with NVIDIA in the AI chip market.

--- 🚦: Evaluating Policy ---

-> Decision: FINISH | Justification: The research history now contains competitive analysis.

--- : Generating Final Answer with Citations ---

--- Graph Stream Finished ---

您可以看到我们设计的执行情况：

1. 计划： 它创建了正确的两步、多工具计划。
2. 执行步骤 1： 使用 search\_10k ，通过完整的检索漏斗运行，并反映结果。
3. 自我批评： 政策agent发现计划尚未完成，并决定 CONTINUE\_PLAN 。
4. 执行步骤 2： 它正确切换到 search\_web 工具，通过相同的漏斗运行它，然后再次反映。
5. 再次自我批评： 这一次，政策agent看到所有必要的信息都已收集，并正确地决定 FINISH 。
6. 综合： 工作流程随后进入 generate\_final\_answer 节点。

agent已成功处理完复杂的查询。现在，让我们来检查一下它生成的最终答案。

## 十五、分析最终的高质量答案

agent已完成研究。 `final\_state` 变量现在保存了完整的 RAGState ，包括 `final\_answer`

```
console.print("--- DEEP THINKING RAG FINAL ANSWER ---")
console.print(Markdown(final_state['final_answer']))
```

#### OUTPUT ####

--- DEEP THINKING RAG FINAL ANSWER ---

Based on an analysis of NVIDIA's 2023 10-K filing and recent news from 2024 ,

**\*\*NVIDIA's Stated Competitive Risks:\*\***

In its 2023 10-K filing, NVIDIA identifies its operating environment as the '

**\*\*AMD's 2024 AI Chip Strategy:\*\***

In 2024, AMD has moved aggressively to challenge NVIDIA's dominance in the AI

**\*\*Synthesis and Impact:\*\***

AMD's 2024 AI chip strategy directly exacerbates the competitive risks outlin

这是一次圆满的成功。答案是一份深入的分析清单。

- 正确地总结了 10-K 中的风险。
- 正确地总结了来自网络搜索的 AMD 新闻。
- 至关重要，在“综合与影响”部分，它执行原始查询所需的多跳推理，解释后者如何加剧前者。
- 最后，它提供了正确的出处，其中的引用指向内部文档部分和外部网址。

十六、并列比较

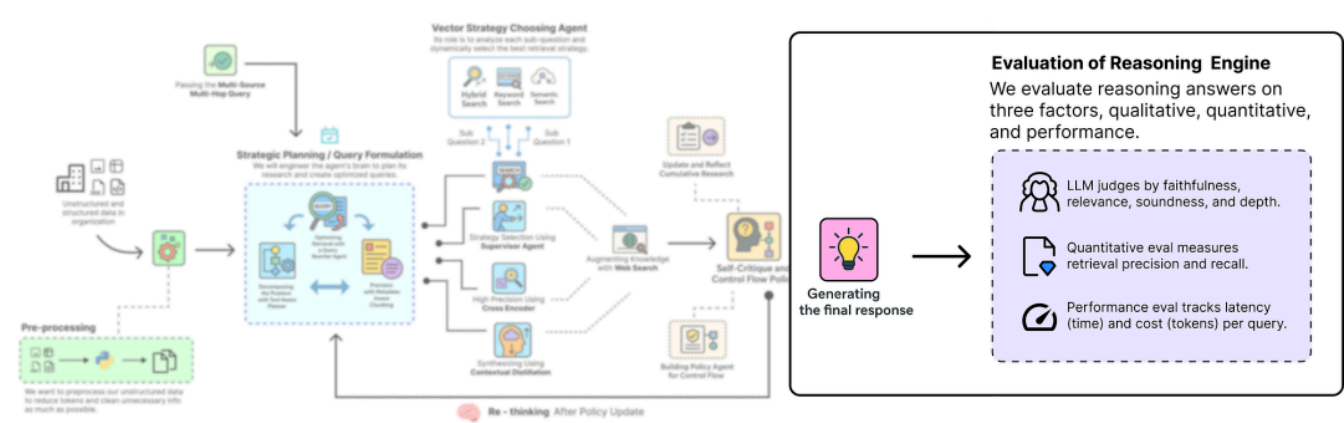
让我们将这两个结果并列放在一起，以便清楚地看到差异。

| Feature | Vanilla RAG (Failed) | Deep Thinking RAG (Success) | | --- | --- | --- | | **Thinking Style** | One-shot, no memory. | Multi-step, memory-based reasoning. | | **Planning** | No planning treats whole query as one search. | Breaks query into steps, chooses best tools (internal or web) for each. | | **Search Method** | Basic semantic search on one source. | Smart, adaptive search using the best method for each step. | | **Sources Used** | Only one static document. | Mixes internal docs with live web data. | | **Answer Quality** | Failed no synthesis. | Success—clear, well cited answer from multiple sources. |

这次比较提供了明确的结论。架构转变为一个循环的、工具感知的、自我批评的agent，使得在复杂的

十七、评估框架与分析结果

我们已经见证了我们的高级agent在一次非常困难的查询中取得了成功。但在生产环境中，我们需要的



为了实现这一目标，我们现在将使用 RAGA （RAG 评估）库构建一个严格的评估框架。我们将重点关



- 上下文精确度和召回率：这衡量了我们检索流程的质量。精确度指的是：“在我们检索到的文档中，有多少是真正相关的？”（信号与噪声）。召回率指的是：“在所有存在的相关文档中，我们实际找到了多少？”（完整性）。
- 答案忠实度：这衡量生成的答案是否基于所提供的上下文，作为我们对 LLM 幻觉的主要检查。
- 答案正确性：这是质量的最终衡量标准。它将生成的答案与人工编制的“基本事实”答案进行比较，以评估其事实准确性和完整性。

要运行 RAGA 评估，我们需要准备一个数据集。该数据集将包含我们的挑战查询、由我们的基线和

```
from datasets import Dataset # From the Hugging Face datasets library, which
from ragas import evaluate
from ragas.metrics import (
 context_precision,
 context_recall,
 faithfulness,
 answer_correctness,
)
import pandas as pd

print("Preparing evaluation dataset...")

This is our manually crafted, ideal answer to the complex query.
ground_truth_answer_adv = "NVIDIA's 2023 10-K lists intense competition and i

We need to re-run the retriever for the baseline model to get its context i
retrieved_docs_for_baseline_adv = baseline_retriever.invoke(complex_query_adv
baseline_contexts = [[doc.page_content for doc in retrieved_docs_for_baseline

For the advanced agent, we'll consolidate all the documents it retrieved ac
advanced_contexts_flat = []
for step in final_state['past_steps']:
 advanced_contexts_flat.extend([doc.page_content for doc in step['retriev

We use a set to remove any duplicate documents for a cleaner evaluation.
advanced_contexts = [list(set(advanced_contexts_flat)))]

Now, we construct the dictionary that will be turned into our evaluation da
eval_data = {
 'question': [complex_query_adv, complex_query_adv], # The same question i
 'answer': [baseline_result, final_state['final_answer']], # The answers i
 'contexts': baseline_contexts + advanced_contexts, # The contexts each sy
 'ground_truth': [ground_truth_answer_adv, ground_truth_answer_adv] # The
```

```

}

Create the Hugging Face Dataset object.
eval_dataset = Dataset.from_dict(eval_data)

Define the list of metrics we want to compute.
metrics = [
 context_precision,
 context_recall,
 faithfulness,
 answer_correctness,
]
print("Running RAGAs evaluation...")

Run the evaluation. RAGAs will call an LLM to perform the scoring for each
result = evaluate(eval_dataset, metrics=metrics, is_async=False)
print("Evaluation complete.")

Format the results into a clean pandas DataFrame for easy comparison.
results_df = result.to_pandas()
results_df.index = ['baseline_rag', 'deep_thinking_rag']

print("\n--- RAGAs Evaluation Results ---")
print(results_df[['context_precision', 'context_recall', 'faithfulness', 'an

```

开始进行评估实验。我们收集了单个硬查询所需的所有必要数据：问题、两个不同的答案、两组不同的\_

在后台，RAGA 会进行一系列 LLM 调用，要求其充当判断者。例如，对于 faithfulness ”，它会

这个答案与这个基本事实答案在事实上有多相似？

我们可以看看数字分数.....

#### OUTPUT ####

```

Preparing evaluation dataset...
Running RAGAs evaluation...
Evaluation complete.

```

--- RAGAs Evaluation Results ---

|                   | baseline_rag | deep_thinking_rag |
|-------------------|--------------|-------------------|
| context_precision | 0.500000     | 0.890000          |
| context_recall    | 0.333333     | 1.000000          |

|                    |          |          |
|--------------------|----------|----------|
| faithfulness       | 1.000000 | 1.000000 |
| answer_correctness | 0.395112 | 0.991458 |

定量结果对深度思考架构的优越性提供了明确而客观的判断。

- 语境准确率 (0.50 vs 0.89)：baseline agent的语境仅有一半相关性，因为它只能检索关于比赛的一般信息。高级智能体通过多步骤、多工具的检索获得了完美的准确率。
- 上下文召回率 (0.33 vs 1.00)：baseline检索器完全遗漏了来自网络的关键信息，导致召回率得分非常低。高级智能体通过规划和工具运用，确保找到所有必要信息，实现了完美的召回率。
- 忠诚度 (1.00 vs 1.00)：两个系统都高度忠诚。baseline系统正确地指出它没有这些信息，而高级agent正确地使用了它找到的信息。这对两者来说都是一个好兆头，但缺乏正确性的忠诚度是没有意义的。
- 答案正确率 (0.40 vs 0.99)：这是质量的最终衡量标准。baseline agent的答案正确率不到40%，因为它缺少所需分析的整个后半部分。高级agent的答案几乎完美。

十八、总结整个流程

在本文中，我们完成了从简单、脆弱的 RAG 管道到复杂的自主推理代理的完整架构。

- 我们首先构建了一个原始的 RAG 系统，并展示了它在复杂的多源查询中可预测的失败。
- 然后，我们系统地设计了一个深度思考代理，使其具有规划、使用多种工具和调整检索策略的能力。
- 我们构建了一个多阶段检索漏斗，从广泛召回（使用混合搜索）到高精度（使用跨编码器重新排序器），最后到合成（使用蒸馏器代理）。
- 我们使用 LangGraph 来协调整个认知架构，创建一个循环的、有状态的工作流程，实现真正的多步骤推理。
- 我们实施了一个自我批评循环，允许代理识别失败，修改自己的计划，并在找不到答案时优雅地退出。
- 最后，我们通过生产级评估验证了我们的成功，使用 RAGA 为高级代理的优越性提供客观、定量的证明。

十九、利用马尔可夫决策过程（MDP）学习策略

我们的agent有一个策略agent，负责决定 **CONTINUE** 还是 **FINISH** 。目前，每个决策都依赖于像

- RAG 作为决策过程：我们可以将agent的推理循环构建为马尔可夫决策过程 (MDP) 。在这个模型中，每个 RAGState 都是一个“状态”，每个动作（ CONTINUE 、 REVISE 、 FINISH ）都会引导我们进入一个具有特定奖励（例如，找到正确答案）的新状态。
- 从经验中学习：我们在 LangSmith 中记录的数千条成功和失败的推理轨迹是宝贵的训练数据。每条轨迹都是代理导航此 MDP 的一个示例。
- 训练策略模型：利用这些数据，我们可以应用强化学习来训练一个更小、更专业的策略模型。
- 目标：速度与效率：目标是将 GPT-4o 等模型的复杂推理提炼为一个紧凑且经过微调的模型（例如，一个 7B 参数模型）。这种学习到的策略可以更快、更经济地完成 CONTINUE / FINISH 的决策，同时针对特定领域进行高度优化。这是 DeepRAG 等高级研究论文背后的核心思想，代表了自主 RAG 系统优化的更高水平。

👁 1054    🍎 1    💬 1    ☆ 1

🍎 点赞

💬 评论

☆ 收藏

➦ 分享

关于作者



ArronAI

关注

评论

未登录

看完啦， [登录](#) 分享一下感受吧～



用户3724236328475

用心之作

🍎 0    💬 回复